Introduction to Software Engineering (LECT 1)

Organization of this Lecture

- What is Software Engineering?
- Programs vs. Software Products
- Evolution of Software Engineering
- Notable Changes In Software Development Practices
- Introduction to Life Cycle Models
- Summary

What is Software Engineering?

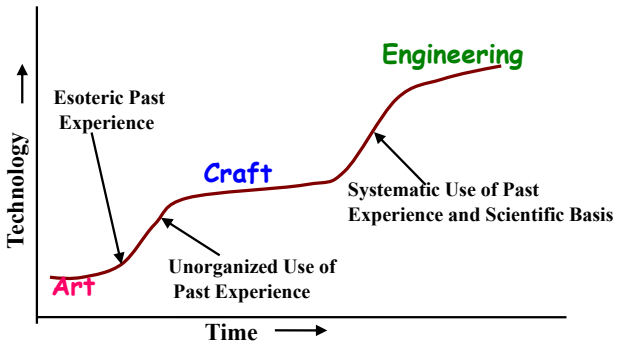
- Engineering approach to develop software.
  - Building Construction Analogy.

- Systematic collection of past experience:
  - Techniques,
  - Methodologies,
  - Guidelines.

Engineering Practice

- Heavy use of past experience:
  - Past experience is systematically arranged.
- Theoretical basis and quantitative techniques provided.
- Many are just thumb rules.
- Tradeoff between alternatives.
- Pragmatic approach to cost-effectiveness.

Technology Development Pattern

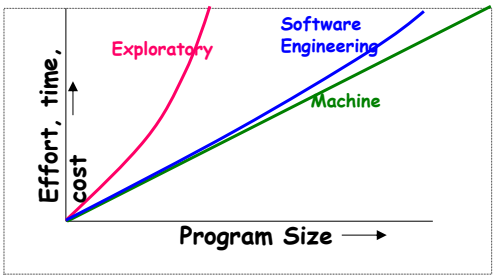


Evolution of an Art into an Engineering Discipline

- The early programmers used an exploratory (also called build and fix) style.
  - In the build and fix (exploratory) style, normally a `dirty' program is quickly developed.
  - The different imperfections that are subsequently noticed are fixed.

What is Wrong with the Exploratory Style?

- Can successfully be used for very small programs only.



What is Wrong with the Exploratory Style? Cont…

- Besides the exponential growth of effort, cost, and time with problem size:
  - Exploratory style usually results in unmaintainable code.
  - It becomes very difficult to use the exploratory style in a team development environment.

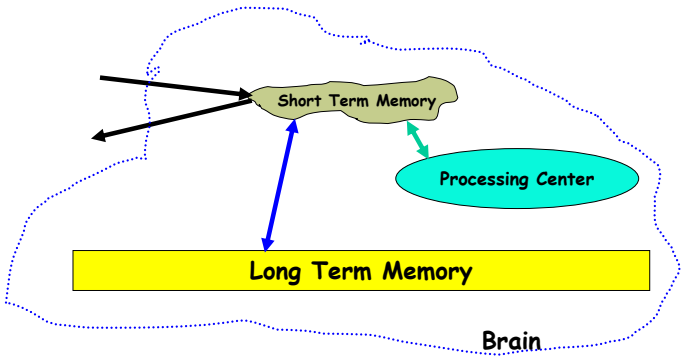
What is Wrong with the Exploratory Style? Cont…

- Why does the effort required to develop a product grow exponentially with product size?
  - Why does the approach completely breaks down when the product size becomes large?

An Interpretation Based on Human Cognition Mechanism

- Human memory can be thought to be made up of two distinct parts [Miller 56]:
  - Short term memory and
  - Long term memory.
- If you are asked the question: ``If it is 10AM now, how many hours are remaining today?"
  - First, 10AM would be stored in the short-term memory.
  - Next, a day is 24 hours long would be fetched from the long term memory into short term memory.
  - Finally, the mental manipulation unit would compute the difference (24-10).

Human Cognition Mechanism



Short Term Memory

- An item stored in the short term memory can get lost:
  - Either due to decay with time or
  - Displacement by newer information.
- This restricts the time for which an item is stored in short term memory to few tens of seconds.
  - However, an item can be retained longer in the short term memory by recycling.

What is an Item?

- An item is any set of related information.
  - A character such as `a' or a digit such as `5' can be items.
  - A word, a sentence, a story, or even a picture can each be a single item.
- Each item normally occupies one place in memory.
- When you are able to relate several different items together (chunking):
  - The information that should normally occupy several places can be stored using only one place in the memory.

Chunking

- If you are given the binary number 110010101001
  - It may prove very hard for you to understand and remember.
  - But, the octal form of 6251 (i.e. (110)(010)(101)(001)) would be easier.
  - You have managed to create chunks of three items each.

Evidence of Short Term Memory

- Short term memory is evident:
  - In many of our day-to-day experiences.
- Suppose, you look up a number from the telephone directory and start dialling it.
  - If you find the number to be busy, you can dial the number again after a few seconds almost effortlessly without having to look up the directory.
- But, after several days:
  - You may not remember the number at all, and would need to consult the directory again.

The Magical Number 7

- If a person deals with seven or less number items:
  - These would be easily be accommodated in the short term memory.
  - So, he can easily understand it.
- As the number of new information increases beyond seven,
  - It becomes exceedingly difficult to understand it.

Implication in Program Development

- A small program having just a few variables:
  - Is within the easy grasp of an individual.
- As the number of independent variables in the program increases:
  - It quickly exceeds the grasping power of an individual:
    - Requires an unduly large effort to master the problem.

Implication in Program Development

- Instead of a human, if a machine could be writing (generating) a program,
  - The slope of the curve would be linear.
- But, why does the effort-size curve become almost linear when software engineering principles are deployed?
  - Software engineering principles extensively use techniques specifically to overcome the human cognitive limitations.

Principles Deployed by Software Engineering to Overcome Human Cognitive Limitations

- Mainly two important principles are deployed:
  - Abstraction
  - Decomposition

Abstraction

- Simplify a problem by omitting unnecessary details.
  - Focus attention on only one aspect of the problem and ignore irrelevant details.
- Suppose you are asked to develop an overall understanding of some country.
  - No one in his right mind would meet all the citizens of the country, visit every house, and examine every tree of the country, etc.
  - You would possibly refer to various types of maps for that country.
- A map, in fact, is an abstract representation of a country.

Decomposition

- Decompose a problem into many small independent parts.
  - The small parts are then taken up one by one and solved separately.
  - The idea is that each small part would be easy to grasp and can be easily solved.
  - The full problem is solved when all the parts are solved.

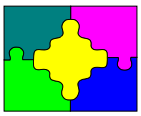
Decomposition

- A popular way to demonstrate the decomposition principle:
  - Try to break a bunch of sticks tied together versus breaking them individually.
- Example use of decomposition principle:
  - You understand a book better when the contents are organized into independent chapters
  - Compared to when everything is mixed up.

Why Study Software Engineering? (1)

- To acquire skills to develop large programs.
  - Exponential growth in complexity and difficulty level with size.
  - The ad hoc approach breaks down when size of software increases.

Why Study Software Engineering? (2)



- Ability to solve complex programming problems:
  - How to break large projects into smaller and manageable parts?
  - How to use abstraction?
- Also learn techniques of:
  - Specification, design, user interface development, testing, project management, etc.

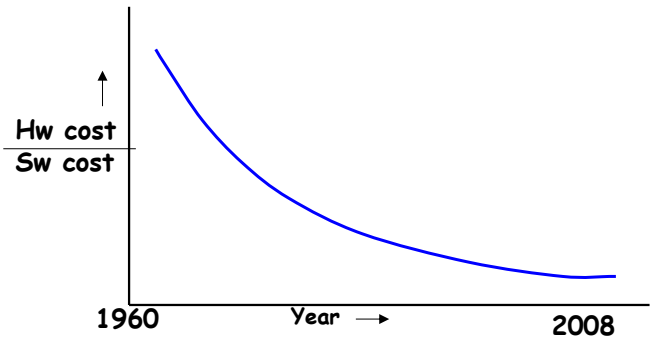
Why Study Software Engineering? (3)

- To acquire skills to be a better programmer:
  - Higher Productivity
  - Better Quality Programs

Software Crisis

- Software products:
  - Fail to meet user requirements.
  - Frequently crash.
  - Expensive.
  - Difficult to alter, debug, and enhance.
  - Often delivered late.
  - Use resources non-optimally.

Software Crisis (cont.)



Relative Cost of Hardware and Software

Factors Contributing to the Software Crisis

- Larger problems,
- Lack of adequate training in software engineering,
- Increasing skill shortage,
- Low productivity improvements.

Programs versus Software Products

Usually small in size	Large
Author himself is sole user	Large number of users
Single developer	Team of developers
Lacks proper user interface	Well-designed interface
Lacks proper documentation	Well documented & user-manual prepared
Ad hoc development.	Systematic development

Types of Software Projects

- Software products
- Outsourced projects
- Indian companies have focused on outsourced projects.

Computer Systems Engineering

- Computer systems engineering:
  - encompasses software engineering.
- Many products require development of software as well as specific hardware to run it:
  - a coffee vending machine,
  - a mobile communication product, etc.

Computer Systems Engineering

- The high-level problem:
  - Deciding which tasks are to be solved by software.
  - Which ones by hardware.

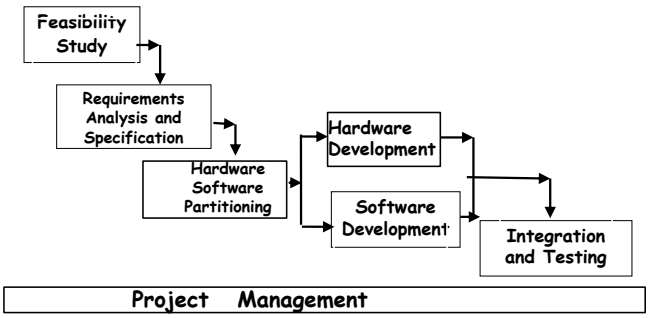
Computer Systems Engineering

(CONT.)

- Often, hardware and software are developed together:
  - Hardware simulator is used during software development.
- Integration of hardware and software.
- Final system testing

Computer Systems Engineering

(CONT.)



Emergence of Software Engineering

- Early Computer Programming (1950s):
  - Programs were being written in assembly language.
  - Programs were limited to about a few hundreds of lines of assembly code.

Early Computer Programming (50s)

- Every programmer developed his own style of writing programs:
  - According to his intuition (exploratory programming).

High-Level Language Programming (Early 60s)

- High-level languages such as FORTRAN, ALGOL, and COBOL were introduced:
  - This reduced software development efforts greatly.

High-Level Language Programming (Early 60s)

- Software development style was still exploratory.
  - Typical program sizes were limited to a few thousands of lines of source code.

Control Flow-Based Design (late 60s)

- Size and complexity of programs increased further:
  - Exploratory programming style proved to be insufficient.
- Programmers found:
  - Very difficult to write cost-effective and correct programs.

Control Flow-Based Design (late 60s)

- Programmers found:
  - programs written by others very difficult to understand and maintain.
- To cope up with this problem, experienced programmers advised: ``Pay particular attention to the design of the program's control structure.''

Control Flow-Based Design (late 60s)

- A program's control structure indicates:
  - The sequence in which the program's instructions are executed.
- To help design programs having good control structure:
  - Flow charting technique was developed.

Control Flow-Based Design (late 60s)

- Using flow charting technique:
  - One can represent and design a program's control structure.
  - Usually one understands a program:
    - By mentally simulating the program's execution sequence.

Control Flow-Based Design (Late 60s)

- A program having a messy flow chart representation:
  - Difficult to understand and debug.

Control Flow-Based Design (Late 60s)

- It was found:
  - GO TO statements makes control structure of a program messy.
  - GO TO statements alter the flow of control arbitrarily.
  - The need to restrict use of GO TO statements was recognized.

Control Flow-Based Design (Late 60s)

- Many programmers had extensively used assembly languages.
  - JUMP instructions are frequently used for program branching in assembly languages.
  - Programmers considered use of GO TO statements inevitable.

Control-flow Based Design (Late 60s)

- At that time, Dijkstra published his article:
  - "Goto Statement Considered Harmful" Comm. of ACM, 1969.
- Many programmers were unhappy to read his article.

Control Flow-Based Design (Late 60s)

- They published several counter articles:
  - Highlighting the advantages and inevitability of GO TO statements.

Control Flow-Based Design (Late 60s)

- But, soon it was conclusively proved:
  - Only three programming constructs are sufficient to express any programming logic:
    - sequence (e.g. a=0;b=5;)
    - selection (e.g.if(c=true) k=5 else m=5;)
    - iteration (e.g. while(k>0) k=j-k;)

Control-flow Based Design (Late 60s)

- Everyone accepted:
  - It is possible to solve any programming problem without using GO TO statements.
  - This formed the basis of Structured Programming methodology.

Structured Programming

- A program is called structured
  - When it uses only the following types of constructs:
    - sequence,
    - selection,
    - iteration

Structured Programs

- Unstructured control flows are avoided.
- Consist of a neat set of modules.
- Use single-entry, single-exit program constructs.

Structured Programs

- However, violations to this feature are permitted:
  - Due to practical considerations such as:
    - Premature loop exit to support exception handling.

Structured programs

- Structured programs are:
  - Easier to read and understand,
  - Easier to maintain,
  - Require less effort and time for development.

Structured Programming

- Research experience shows:
  - Programmers commit less number of errors:
    - While using structured if-then-else and do-while statements.
    - Compared to test-and-branch constructs.

Data Structure-Oriented Design (Early 70s)

- Soon it was discovered:
  - It is important to pay more attention to the design of data structures of a program
    - Than to the design of its control structure.

Data Structure-Oriented Design (Early 70s)

- Techniques which emphasize designing the data structure:
  - Derive program structure from it:
    - Are called data structure-oriented design techniques.

Data Structure Oriented Design (Early 70s)

- Example of data structure-oriented design technique:
  - Jackson's Structured Programming(JSP) methodology
    - Developed by Michael Jackson in 1970s.

Data Structure Oriented Design (Early 70s)

- JSP technique:
  - Program code structure should correspond to the data structure.

Data Structure Oriented Design (Early 70s)

- In JSP methodology:
  - A program's data structures are first designed using notations for
    - sequence, selection, and iteration.
  - Then data structure design is used :
    - To derive the program structure.

Data Structure Oriented Design (Early 70s)

- Several other data structure-oriented Methodologies also exist:
  - e.g., Warnier-Orr Methodology.

Data Flow-Oriented Design

(Late 70s)

- Data flow-oriented techniques advocate:
  - The data items input to a system must first be identified,
  - Processing required on the data items to produce the required outputs should be determined.

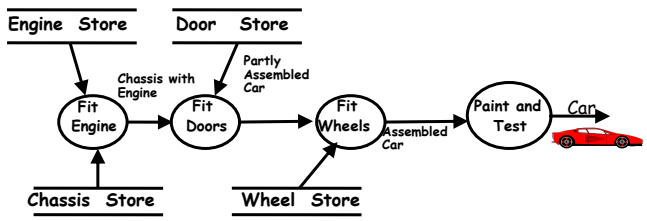
Data Flow-Oriented Design (Late 70s)

- Data flow technique identifies:
  - Different processing stations (functions) in a system.
  - The items (data) that flow between processing stations.

Data Flow-Oriented Design (Late 70s)

- Data flow technique is a generic technique:
  - Can be used to model the working of any system.
    - not just software systems.
- A major advantage of the data flow technique is its simplicity.

Data Flow Model of a Car Assembly Unit



Object-Oriented Design (80s)

- Object-oriented technique:
  - An intuitively appealing design approach:
  - Natural objects (such as employees, pay-roll-register, etc.) occurring in a problem are first identified.

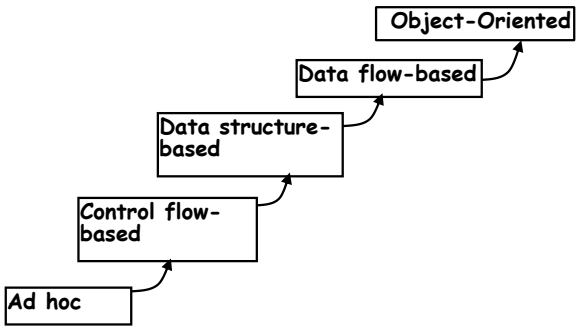
Object-Oriented Design (80s)

- Relationships among objects:
  - Such as composition, reference, and inheritance are determined.
- Each object essentially acts as
  - A data hiding (or data abstraction) entity.

Object-Oriented Design (80s)

- Object-Oriented Techniques have gained wide acceptance:
  - Simplicity
  - Reuse possibilities
  - Lower development time and cost
  - More robust code
  - Easy maintenance

Evolution of Design Techniques



Evolution of Other Software Engineering Techniques

- The improvements to the software design methodologies
  - are indeed very conspicuous.
- In additions to the software design techniques:
  - Several other techniques evolved.

Evolution of Other Software Engineering Techniques

- life cycle models,
- specification techniques,
- project management techniques,
- testing techniques,
- debugging techniques,
- quality assurance techniques,
- software measurement techniques,
- CASE tools, etc.

Differences between the exploratory style and modern software development practices

- Use of Life Cycle Models
- Software is developed through several well-defined stages:
  - requirements analysis and specification,
  - design,
  - coding,
  - testing, etc.

Differences between the exploratory style and modern software development practices

- Emphasis has shifted
  - from error correction to error prevention.
- Modern practices emphasize:
  - detection of errors as close to their point of introduction as possible.

Differences between the exploratory style and modern software development practices (CONT.)

- In exploratory style,
  - errors are detected only during testing,
- Now,
  - focus is on detecting as many errors as possible in each phase of development.

Differences between the exploratory style and modern software development practices (CONT.)

- In exploratory style,
  - coding is synonymous with program development.
- Now,
  - coding is considered only a small part of program development effort.

Differences between the exploratory style and modern software development practices (CONT.)

- A lot of effort and attention is now being paid to:
  - Requirements specification.
- Also, now there is a distinct design phase:
  - Standard design techniques are being used.

Differences between the exploratory style and modern software development practices (CONT.)

- During all stages of development process:
  - Periodic reviews are being carried out
- Software testing has become systematic:
  - Standard testing techniques are available.

Differences between the exploratory style and modern software development practices (CONT.)

- There is better visibility of design and code:
  - Visibility means production of good quality, consistent and standard documents.
  - In the past, very little attention was being given to producing good quality and consistent documents.
  - We will see later that increased visibility makes software project management easier.

Differences between the exploratory style and modern software development practices (CONT.)

- Because of good documentation:
  - fault diagnosis and maintenance are smoother now.
- Several metrics are being used:
  - help in software project management, quality assurance, etc.

Differences between the exploratory style and modern software development practices (CONT.)

- Projects are being thoroughly planned:
  - estimation,
  - scheduling,
  - monitoring mechanisms.
- Use of CASE tools.

Software Life Cycle

- Software life cycle (or software process):
  - Series of identifiable stages that a software product undergoes during its life time:
    - Feasibility study
    - Requirements analysis and specification,
    - Design,
    - Coding,
    - Testing
    - maintenance.

Life Cycle Model

- A software life cycle model (or process model):
  - a descriptive and diagrammatic model of software life cycle:
  - identifies all the activities required for product development,
  - establishes a precedence ordering among the different activities,
  - Divides life cycle into phases.

Life Cycle Model (CONT.)

- Several different activities may be carried out in each life cycle phase.
  - For example, the design stage might consist of:
    - structured analysis activity followed by
    - structured design activity.

Why Model Life Cycle ?

- A written description:
  - Forms a common understanding of activities among the software developers.
  - Helps in identifying inconsistencies, redundancies, and omissions in the development process.
  - Helps in tailoring a process model for specific projects.

Why Model Life Cycle ?

- Processes are tailored for special projects.
  - A documented process model
    - Helps to identify where the tailoring is to occur.

Life Cycle Model (CONT.)

- The development team must identify a suitable life cycle model:
  - and then adhere to it.
  - Primary advantage of adhering to a life cycle model:
    - Helps development of software in a systematic and disciplined manner.

Life Cycle Model (CONT.)

- When a program is developed by a single programmer ---
  - he has the freedom to decide his exact steps.

Life Cycle Model (CONT.)

- When a software product is being developed by a team:
  - there must be a precise understanding among team members as to when to do what,
  - otherwise it would lead to chaos and project failure.

Life Cycle Model (CONT.)

- A software project will never succeed if:
  - one engineer starts writing code,
  - another concentrates on writing the test document first,
  - yet another engineer first defines the file structure
  - another defines the I/O for his portion first.

Life Cycle Model (CONT.)

- A life cycle model:
  - defines entry and exit criteria for every phase.
  - A phase is considered to be complete:
    - only when all its exit criteria are satisfied.

Life Cycle Model (CONT.)

- The phase exit criteria for the software requirements specification phase:
  - Software Requirements Specification (SRS) document is complete, reviewed, and approved by the customer.
- A phase can start:
  - only if its phase-entry criteria have been satisfied.

Life Cycle Model (CONT.)

- It becomes easier for software project managers:
  - to monitor the progress of the project.

Life Cycle Model (CONT.)

- When a life cycle model is adhered to,
  - the project manager can at any time fairly accurately tell,
    - at which stage (e.g., design, code, test, etc.) of the project is.
  - Otherwise, it becomes very difficult to track the progress of the project
    - the project manager would have to depend on the guesses of the team members.

Life Cycle Model (CONT.)

- This usually leads to a problem:
  - known as the 99% complete syndrome.

Life Cycle Model (CONT.)

- Many life cycle models have been proposed.
- We will confine our attention to a few important and commonly used models.
  - Classical waterfall model
  - Iterative waterfall,
  - Evolutionary,
  - Prototyping, and
  - Spiral model

Summary

- Software engineering is:
  - Systematic collection of decades of programming experience
  - Together with the innovations made by researchers.

Summary

- A fundamental necessity while developing any large software product:
  - Adoption of a life cycle model.

Summary

- **Adherence to a software life cycle model:**
  - **Helps to do various development activities in a systematic and disciplined manner.**
  - **Also makes it easier to manage a software development effort.**

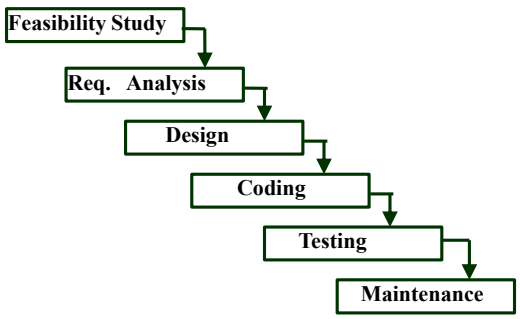
Life Cycle Models

(Lecture 2)

Classical Waterfall Model

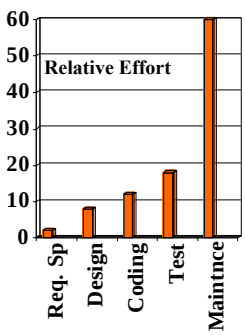
- Classical waterfall model divides life cycle into phases:
  - feasibility study,
  - requirements analysis and specification,
  - design,
  - coding and unit testing,
  - integration and system testing,
  - maintenance.

Classical Waterfall Model



Relative Effort for Phases

- Phases between feasibility study and testing
  - known as development phases.
- Among all life cycle phases
  - maintenance phase consumes maximum effort.
- Among development phases,
  - testing phase consumes the maximum effort.



Classical Waterfall Model

(CONT.)

- Most organizations usually define:
  - **standards on the outputs (deliverables) produced at the end of every phase**
  - **entry and exit criteria for every phase.**
- They also prescribe specific methodologies for:
  - **specification,**
  - **design,**
  - **testing,**
  - **project management, etc.**

Classical Waterfall Model
(CONT.)

- The guidelines and methodologies of an organization:
  - **called the organization's software development methodology.**
- Software development organizations:
  - **expect fresh engineers to master the organization's software development methodology.**

Feasibility Study

- Main aim of feasibility study: determine whether developing the product
  - financially worthwhile
  - technically feasible.
- First roughly understand what the customer wants:
  - different data which would be input to the system,
  - processing needed on these data,
  - output data to be produced by the system,
  - various constraints on the behavior of the system.

Activities during Feasibility Study

- Work out an overall understanding of the problem.
- Formulate different solution strategies.
- Examine alternate solution strategies in terms of:
  - resources required,
  - cost of development, and
  - development time.

Activities during Feasibility Study

- Perform a cost/benefit analysis:
  - to determine which solution is the best.
  - you may determine that none of the solutions is feasible due to:
    - high cost,
    - resource constraints,
    - technical reasons.

Requirements Analysis and Specification

- Aim of this phase:
  - understand the exact requirements of the customer,
  - document them properly.
- Consists of two distinct activities:
  - requirements gathering and analysis
  - requirements specification.

Goals of Requirements Analysis

- Collect all related data from the customer:
  - analyze the collected data to clearly understand what the customer wants,
  - find out any inconsistencies and incompleteness in the requirements,
  - resolve all inconsistencies and incompleteness.

Requirements Gathering

- Gathering relevant data:
  - usually collected from the end-users through interviews and discussions.
  - For example, for a business accounting software:
    - interview all the accountants of the organization to find out their requirements.

Requirements Analysis (CONT.)

- The data you initially collect from the users:
  - would usually contain several contradictions and ambiguities:
  - each user typically has only a partial and incomplete view of the system.

Requirements Analysis (CONT.)

- Ambiguities and contradictions:
  - must be identified
  - resolved by discussions with the customers.
- Next, requirements are organized:
  - into a Software Requirements Specification (SRS) document.

Requirements Analysis (CONT.)

- Engineers doing requirements analysis and specification:
  - are designated as analysts.

Design

- Design phase transforms requirements specification:
  - into a form suitable for implementation in some programming language.

Design

- In technical terms:
  - during design phase, software architecture is derived from the SRS document.
- Two design approaches:
  - traditional approach,
  - object oriented approach.

Traditional Design Approach

- **Consists of two activities:**
  - **Structured analysis**
  - **Structured design**

Structured Analysis Activity

- Identify all the functions to be performed.
- Identify data flow among the functions.
- Decompose each function recursively into sub-functions.
  - Identify data flow among the subfunctions as well.

Structured Analysis (CONT.)

- Carried out using Data flow diagrams (DFDs).
- After structured analysis, carry out structured design:
  - architectural design (or high-level design)
  - detailed design (or low-level design).

Structured Design

- High-level design:
  - decompose the system into *modules*,
  - represent invocation relationships among the modules.
- Detailed design:
  - different modules designed in greater detail:
    - data structures and algorithms for each module are designed.

Object Oriented Design

- First identify various objects (real world entities) occurring in the problem:
  - identify the relationships among the objects.
  - For example, the objects in a pay-roll software may be:
    - employees,
    - managers,
    - pay-roll register,
    - Departments, etc.

Object Oriented Design (CONT.)

- Object structure
  - further refined to obtain the detailed design.
- OOD has several advantages:
  - lower development effort,
  - lower development time,
  - better maintainability.

Implementation

- Purpose of implementation phase (aka coding and unit testing phase):
  - translate software design into source code.

Implementation

- During the implementation phase:
  - each module of the design is coded,
  - each module is unit tested
    - tested independently as a stand alone unit, and debugged,
  - each module is documented.

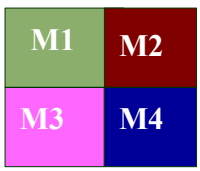
Implementation (CONT.)

- The purpose of unit testing:
  - test if individual modules work correctly.
- The end product of implementation phase:
  - a set of program modules that have been tested individually.

Integration and System Testing

- Different modules are integrated in a planned manner:
  - modules are almost never integrated in one shot.
  - Normally integration is carried out through a number of steps.
- During each integration step,
  - the partially integrated system is tested.

Integration and System Testing



System Testing

- After all the modules have been successfully integrated and tested:
  - system testing is carried out.
- Goal of system testing:
  - ensure that the developed system functions according to its requirements as specified in the SRS document.

Maintenance

- Maintenance of any software product:
  - **requires much more effort than the effort to develop the product itself.**
  - **development effort to maintenance effort is typically 40:60.**

Maintenance (CONT.)

- Corrective maintenance:
  - Correct errors which were not discovered during the product development phases.
- Perfective maintenance:
  - Improve implementation of the system
  - enhance functionalities of the system.
- Adaptive maintenance:
  - Port software to a new environment,
    - e.g. to a new computer or to a new operating system.

Iterative Waterfall Model

- Classical waterfall model is idealistic:
  - assumes that no defect is introduced during any development activity.
  - in practice:
    - defects do get introduced in almost every phase of the life cycle.

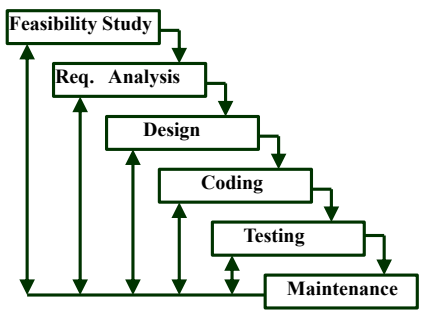
Iterative Waterfall Model (CONT.)

- Defects usually get detected much later in the life cycle:
  - **For example, a design defect might go unnoticed till the coding or testing phase.**

Iterative Waterfall Model (CONT.)

- Once a defect is detected:
  - we need to go back to the phase where it was introduced
  - redo some of the work done during that and all subsequent phases.
- Therefore we need feedback paths in the classical waterfall model.

Iterative Waterfall Model (CONT.)



Iterative Waterfall Model
(CONT.)

- Errors should be detected
  - **in the same phase in which they are introduced.**
- For example:
  - if a design problem is detected in the design phase itself,
    - **the problem can be taken care of much more easily**
    - **than say if it is identified at the end of the integration and system testing phase.**

Phase containment of errors

- Reason: rework must be carried out not only to the design but also to code and test phases.
- The principle of detecting errors as close to its point of introduction as possible:
  - is known as **phase containment of errors.**
- Iterative waterfall model is by far the most widely used model.
  - Almost every other model is derived from the waterfall model.

Classical Waterfall Model
(CONT.)

- Irrespective of the life cycle model actually followed:
  - the documents should reflect a classical waterfall model of development,
  - comprehension of the documents is facilitated.

Classical Waterfall Model
(CONT.)

- Metaphor of mathematical theorem proving:
  - A mathematician presents a proof as a single chain of deductions,
    - even though the proof might have come from a convoluted set of partial attempts, blind alleys and backtracks.

Prototyping Model

- Before starting actual development,
  - a working prototype of the system should first be built.
- A prototype is a toy implementation of a system:
  - limited functional capabilities,
  - low reliability,
  - inefficient performance.

Reasons for developing a prototype

- **Illustrate to the customer:**
  - **input data formats, messages, reports, or interactive dialogs.**
- **Examine technical issues associated with product development:**
  - **Often major design decisions depend on issues like:**
    - **response time of a hardware controller,**
    - **efficiency of a sorting algorithm, etc.**

Prototyping Model (CONT.)

- The third reason for developing a prototype is:
  - it is impossible to ``get it right'' the first time,
  - we must plan to throw away the first product
    - if we want to develop a good product.

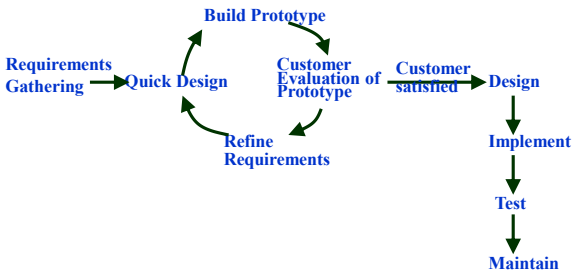
Prototyping Model (CONT.)

- Start with approximate requirements.
- Carry out a quick design.
- Prototype model is built using several short-cuts:
  - **Short-cuts might involve using inefficient, inaccurate, or dummy functions.**
    - **A function may use a table look-up rather than performing the actual computations.**

Prototyping Model (CONT.)

- The developed prototype is submitted to the customer for his evaluation:
  - **Based on the user feedback, requirements are refined.**
  - **This cycle continues until the user approves the prototype.**
- The actual system is developed using the classical waterfall approach.

Prototyping Model (CONT.)



Prototyping Model (CONT.)

- Requirements analysis and specification phase becomes redundant:
  - final working prototype (with all user feedbacks incorporated) serves as an animated requirements specification.
- Design and code for the prototype is usually thrown away:
  - However, the experience gathered from developing the prototype helps a great deal while developing the actual product.

Prototyping Model (CONT.)

- Even though construction of a working prototype model involves additional cost --- overall development cost might be lower for:
  - **systems with unclear user requirements,**
  - **systems with unresolved technical issues.**
- Many user requirements get properly defined and technical issues get resolved:
  - **these would have appeared later as change requests and resulted in incurring massive redesign costs.**

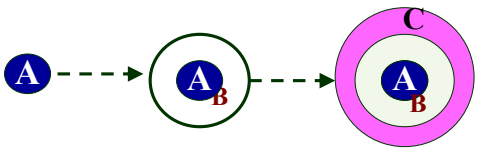
Evolutionary Model

- Evolutionary model (aka successive versions or incremental model):
  - The system is broken down into several modules which can be incrementally implemented and delivered.
- First develop the core modules of the system.
- The initial product skeleton is refined into increasing levels of capability:
  - by adding new functionalities in successive versions.

Evolutionary Model (CONT.)

- Successive version of the product:
  - functioning systems capable of performing some useful work.
  - A new release may include new functionality:
    - also existing functionality in the current release might have been enhanced.

Evolutionary Model (CONT.)



Advantages of Evolutionary Model

- Users get a chance to experiment with a partially developed system:
  - much before the full working version is released,
- Helps finding exact user requirements:
  - much before fully working system is developed.
- Core modules get tested thoroughly:
  - reduces chances of errors in final product.

Disadvantages of Evolutionary Model

- Often, difficult to subdivide problems into functional units:
  - which can be incrementally implemented and delivered.
  - evolutionary model is useful for very large problems,
    - where it is easier to find modules for incremental implementation.

Evolutionary Model with Iteration

- Many organizations use a combination of iterative and incremental development:
  - a new release may include new functionality
  - existing functionality from the current release may also have been modified.

Evolutionary Model with iteration

- Several advantages:
  - Training can start on an earlier release
    - customer feedback taken into account
  - Markets can be created:
    - for functionality that has never been offered.
  - Frequent releases allow developers to fix unanticipated problems quickly.

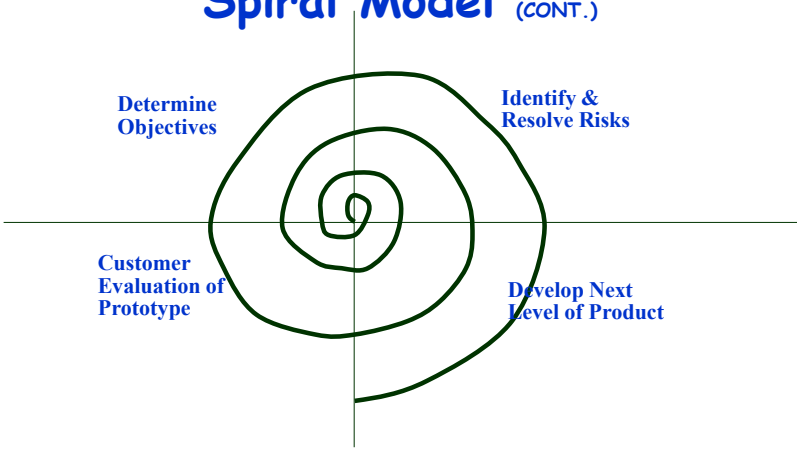
Spiral Model

- Proposed by Boehm in 1988.
- Each loop of the spiral represents a phase of the software process:
  - the innermost loop might be concerned with system feasibility,
  - the next loop with system requirements definition,
  - the next one with system design, and so on.
- There are no fixed phases in this model, the phases shown in the figure are just examples.

Spiral Model (CONT.)

- The team must decide:
  - how to structure the project into phases.
- Start work using some generic model:
  - add extra phases
    - for specific projects or when problems are identified during a project.
- Each loop in the spiral is split into four sectors (quadrants).

Spiral Model (CONT.)



Objective Setting (First Quadrant)

- Identify objectives of the phase,
- Examine the risks associated with these objectives.
  - Risk:
    - any adverse circumstance that might hamper successful completion of a software project.
- Find alternate solutions possible.

Risk Assessment and Reduction (Second Quadrant)

- For each identified project risk,
  - a detailed analysis is carried out.
- Steps are taken to reduce the risk.
- For example, if there is a risk that the requirements are inappropriate:
  - a prototype system may be developed.

Spiral Model (CONT.)

- Development and Validation (Third quadrant):
  - develop and validate the next level of the product.
- Review and Planning (Fourth quadrant):
  - review the results achieved so far with the customer and plan the next iteration around the spiral.
- With each iteration around the spiral:
  - progressively more complete version of the software gets built.

Spiral Model as a meta model

- Subsumes all discussed models:
  - **a single loop spiral represents waterfall model.**
  - **uses an evolutionary approach --**
    - **iterations through the spiral are evolutionary levels.**
  - **enables understanding and reacting to risks during each iteration along the spiral.**
  - **uses:**
    - **prototyping as a risk reduction mechanism**
    - **retains the step-wise approach of the waterfall model**

Comparison of Different Life Cycle Models

- Iterative waterfall model
  - **most widely used model.**
  - **But, suitable only for well-understood problems.**
- Prototype model is suitable for projects not well understood:
  - user requirements
  - technical aspects

Comparison of Different Life Cycle Models (CONT.)

- Evolutionary model is suitable for large problems:
  - **can be decomposed into a set of modules that can be incrementally implemented,**
  - **incremental delivery of the system is acceptable to the customer.**
- The spiral model:
  - **suitable for development of technically challenging software products that are subject to several kinds of risks.**

Requirements Analysis and Specification (Lecture 3)

Organization of this Lecture

- **Introduction**
- **Requirements analysis**
- **Requirements specification**
- **SRS document**
- **Decision table**
- **Decision tree**
- **Summary**

Requirements Analysis and Specification

- Many projects fail:
  - Because they start implementing the system.
  - Without determining whether they are building what the customer really wants.

Requirements Analysis and Specification

- It is important to learn:
  - Requirements analysis and specification techniques carefully.

Requirements Analysis and Specification

- Goals of requirements analysis and specification phase:
  - Fully understand the user requirements.
  - Remove inconsistencies, anomalies, etc. from requirements.
  - Document requirements properly in an SRS document.

Requirements Analysis and Specification

- Consists of two distinct activities:
  - Requirements Gathering and Analysis
  - Specification

Who Carries Out Requirements Analysis and Specification?

- The person who undertakes requirements analysis and specification:
  - Known as systems analyst:
  - Collects data pertaining to the product
  - Analyzes collected data:
    - To understand what exactly needs to be done.
  - Writes the Software Requirements Specification (SRS) document.

Requirements Analysis and Specification

- Final output of this phase:
  - Software Requirements Specification (SRS) Document.
- The SRS document is reviewed by the customer.
  - Reviewed SRS document forms the basis of all future development activities.

Requirements Analysis

- Requirements analysis consists of two main activities:
  - Requirements gathering
  - Analysis of the gathered requirements

Requirements Analysis

- Analyst gathers requirements through:
  - Observation of existing systems,
  - Studying existing procedures,
  - Discussion with the customer and end-users,
  - Analysis of what needs to be done, etc.

Requirements Gathering

- Also known as requirements elicitation.
- If the project is to automate some existing procedures
  - e.g., automating existing manual accounting activities,
  - The task of the system analyst is a little easier
  - Analyst can immediately obtain:
    - input and output formats
    - accurate details of the operational procedures

Requirements Gathering Activities

- 1. Studying the existing documentation
- 2. Interview
- 3. Task analysis
- 4. Scenario analysis
- 5. Form analysis

Requirements Gathering (CONT.)

- In the absence of a working system,
  - Lot of imagination and creativity are required.
- Interacting with the customer to gather relevant data:
  - Requires a lot of experience.

Requirements Gathering (CONT.)

- Some desirable attributes of a good system analyst:
  - Good interaction skills,
  - Imagination and creativity,
  - Experience.

Case Study: Automation of Office Work at CSE Dept.

- The academic, inventory, and financial information at the CSE department:
  - Being carried though manual processing by two office clerks, a store keeper, and two attendants.
- Considering the low budget he had at his
- Disposal:
  - The HoD entrusted the work to a team of student volunteers.

Case Study: Automation of Office Work at CSE Dept.

- The team was first briefed by the HoD about the specific activities to be automated.
- The analyst first discussed with the two clerks:
  - Regarding their specific responsibilities (tasks) that were to be automated.
- The analyst also interviewed student and faculty representatives who would also use the software.

Case Study: Automation of Office Work at CSE Dept.

- For each task, they asked:
  - About the steps through which these are performed.
  - They also discussed various scenarios that might arise for each task.
  - The analyst collected all types of forms that were being used.

Analysis of the Gathered Requirements

- Main purpose of requirements analysis:
  - Clearly understand the user requirements,
  - Detect inconsistencies, ambiguities, and incompleteness.
- Incompleteness and inconsistencies:
  - Resolved through further discussions with the end-users and the customers.

Inconsistent Requirement

- Some part of the requirement:
  - contradicts with some other part.
- <u>Example:</u>
  - One customer says turn off heater and open water shower when temperature > 100 C
  - Another customer says turn off heater and turn ON cooler when temperature > 100 C

Incomplete Requirement

- Some requirements have been omitted:
  - Possibly due to oversight.
- <u>Example:</u>
  - The analyst has not recorded: when temperature falls below 90 C
    - heater should be turned ON
    - water shower turned OFF.

Analysis of the Gathered Requirements (CONT.)

- Requirements analysis involves:
  - Obtaining a clear, in-depth understanding of the product to be developed,
  - Remove all ambiguities and inconsistencies from the initial customer perception of the problem.

Analysis of the Gathered Requirements (CONT.)

- It is quite difficult to obtain:
  - A clear, in-depth understanding of the problem:
    - Especially if there is no working model of the problem.

Analysis of the Gathered Requirements (CONT.)

- Experienced analysts take considerable time:
  - To understand the exact requirements the customer has in his mind.

Analysis of the Gathered Requirements (CONT.)

- Experienced systems analysts know - often as a result of painful experiences ---
  - Without a clear understanding of the problem, it is impossible to develop a satisfactory system.

Analysis of the Gathered Requirements(CONT.)

- Several things about the project should be clearly understood by the analyst:
  - What is the problem?
  - Why is it important to solve the problem?
  - What are the possible solutions to the problem?
  - What complexities might arise while solving the problem?

Analysis of the Gathered Requirements(CONT.)

- Some anomalies and inconsistencies can be very subtle:
  - Escape even most experienced eyes.
  - If a formal model of the system is constructed,
    - Many of the subtle anomalies and inconsistencies get detected.

Analysis of the Gathered Requirements(CONT.)

- After collecting all data regarding the system to be developed,
  - Remove all inconsistencies and anomalies from the requirements,
  - Systematically organize requirements into a Software Requirements Specification (SRS) document.

Software Requirements Specification

- Main aim of requirements specification:
  - Systematically organize the requirements arrived during requirements analysis.
  - Document requirements properly.

Software Requirements Specification

- The SRS document is useful in various contexts:
  - Statement of user needs
  - Contract document
  - Reference document
  - Definition for implementation

Software Requirements Specification: A Contract Document

- Requirements document is a reference document.
- SRS document is a contract between the development team and the customer.
  - Once the SRS document is approved by the customer,
    - Any subsequent controversies are settled by referring the SRS document.

Software Requirements Specification: A Contract Document

- Once customer agrees to the SRS document:
  - Development team starts to develop the product according to the requirements recorded in the SRS document.
- The final product will be acceptable to the customer:
  - As long as it satisfies all the requirements recorded in the SRS document.

SRS Document (CONT.)

- The SRS document is known as <u>black-box specification:</u>
  - The system is considered as a black box whose internal details are not known.
  - Only its visible external (i.e. input/output) behavior is documented.

SRS Document (CONT.)

- SRS document concentrates on:
  - What needs to be done
  - Carefully avoids the solution ("how to do") aspects.
- The SRS document serves as a contract
  - Between development team and the customer.
  - Should be carefully written

SRS Document (CONT.)

- The requirements at this stage:
  - Written using end-user terminology.
- If necessary:
  - Later a formal requirement specification may be developed from it.

Properties of a Good SRS Document

- It should be concise
  - and at the same time should not be ambiguous.
- It should specify what the system must do
  - and not say how to do it.
- Easy to change.,
  - i.e. it should be well-structured.
- It should be consistent.
- It should be complete.

Properties of a Good SRS Document (cont...)

- It should be traceable
  - You should be able to trace which part of the specification corresponds to which part of the design, code, etc and vice versa.
- It should be verifiable
  - e.g. "system should be user friendly" is not verifiable

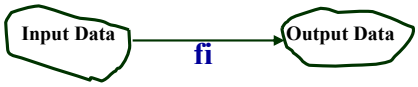
SRS Document (CONT.)

- SRS document, normally contains three important parts:
  - Functional requirements,
  - Non-functional requirements,
  - Goals of Implementation.

- An SRS document should clearly document the following aspects of a software:
- Functional requirements
- Non-functional requirements
  - Design and implementation constraints
  - External interfaces required
  - Other non-functional requirements
- Goals of implementation.

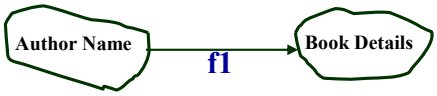
SRS Document (CONT.)

- It is desirable to consider every system:
  - Performing a set of functions {fi}.
  - Each function fi considered as:
  - Transforming a set of input data to corresponding output data.



Example: Functional Requirement

- F1: Search Book
  - Input:
    - an author's name:
  - Output:
    - details of the author's books and the locations of these books in the library.



Functional Requirements

- Functional requirements describe:
  - A set of high-level requirements
  - Each high-level requirement:
    - takes in some data from the user
    - outputs some data to the user
  - Each high-level requirement:
    - might consist of a set of identifiable functions

Functional Requirements

- For each high-level requirement:
  - Every function is described in terms of:
    - Input data set
    - Output data set
    - Processing required to obtain the output data set from the input data set.

Nonfunctional Requirements

- Characteristics of the system which can not be expressed as functions:
  - Maintainability,
  - Portability,
  - Usability, etc.

Nonfunctional Requirements

- Nonfunctional requirements include:
  - Reliability issues,
  - Performance issues:
    - Example: How fast the system can produce results
      - so that it does not overload another system to which it supplies data, etc.
  - Human-computer interface issues,
  - Interface with other external systems,
  - Security, maintainability, etc.

Non-Functional Requirements

- Hardware to be used,
- Operating system
  - or DBMS to be used
- Capabilities of I/O devices
- Standards compliance
- Data representations
  - by the interfaced system

Goals of Implementation

- Goals describe things that are desirable of the system:
  - But, would not be checked for compliance.
  - For example,
    - Reusability issues
    - Functionalities to be developed in future

Organization of the SRS Document

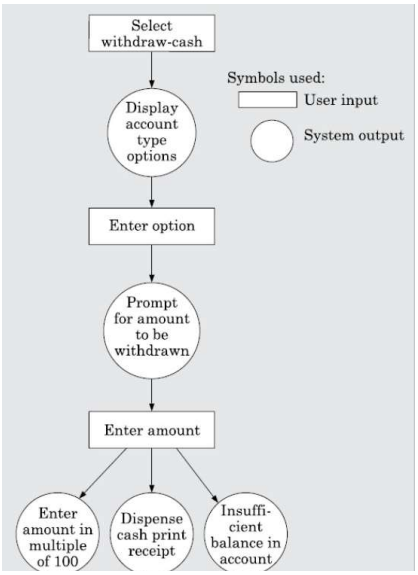
- An SRS document should clearly document the following aspects of a software:
- Functional requirements
- Non-functional requirements
  - Design and implementation constraints
  - External interfaces required
  - Other non-functional requirements
- Goals of implementation.

Functional Requirements

- A high-level function is one:
  - Using which the user can get some useful piece of work done.
- Can the receipt printing work during withdrawal of money from an ATM:
  - Be called a useful piece of work?
- A high-level requirement typically involves:
  - Accepting some data from the user,
  - Transforming it to the required response, and then
  - Outputting the system response to the user.

High-Level Function

- A high-level function:
  - Usually involves a series of interactions between the system and one or more users.
- Even for the same high-level function,
  - There can be different interaction sequences (or scenarios)
  - Due to users selecting different options or entering different data items.



Example Functional Requirements

- List all functional requirements
  - with proper numbering.
- Req. 1:
  - Once the user selects the "search" option,
    - he is asked to enter the key words.
  - The system should output details of all books
    - whose title or author name matches any of the key words entered.
    - Details include: Title, Author Name, Publisher name, Year of Publication, ISBN Number, Catalog Number, Location in the Library.

Example Functional Requirements

- Req. 2:
  - When the "renew" option is selected,
    - The user is asked to enter his membership number and password.
  - After password validation,
    - The list of the books borrowed by him are displayed.
  - The user can renew any of the books:
    - By clicking in the corresponding renew box.

Req. 1:

- R.1.1:
  - Input: "search" option,
  - Output: user prompted to enter the key words.
- R1.2:
  - Input: key words
  - Output: Details of all books whose title or author name matches any of the key words.
    - Details include: Title, Author Name, Publisher name, Year of Publication, ISBN Number, Catalog Number, Location in the Library.
  - Processing: Search the book list for the keywords

Req. 2:

- R2.1:
  - Input: "renew" option selected,
  - Output: user prompted to enter his membership number and password.
- R2.2:
  - Input: membership number and password
  - Output:
    - list of the books borrowed by user are displayed. User prompted to enter books to be renewed or
    - user informed about bad password
  - Processing: Password validation, search books issued to the user from borrower list and display.

Req. 2:

- R2.3:
  - Input: user choice for renewal of the books issued to him through mouse clicks in the corresponding renew box.
  - Output: Confirmation of the books renewed
  - Processing: Renew the books selected by the in the borrower list.

Design and implementation constraints:

- Design and implementation constraints are an important category of non-functional requirements describe any items or issues that will limit the options available to the developers.
- Some of the example constraints can be—corporate or regulatory policies that needs to be honoured; hardware limitations; interfaces with other applications; specific technologies, tools, and databases to be used; specific communications protocols to be used; security considerations; design conventions or programming standards to be followed, etc.

External interfaces required

Examples of external interfaces are—

- hardware, software and communication interfaces, user interfaces, report formats, etc. To specify the user interfaces, each interface between the software and the users must be described. The description may include sample screen images, any GUI standards or style guides

Other non-functional requirements:

- This section contains a description of non-functional requirements that are neither design constraints and nor are external interface requirements. An important example is a performance requirement such as the number of transactions completed per unit time.

Goals of implementation

- The 'goals of implementation' part of the SRS document offers some general suggestions regarding the software to be developed. These are not binding on the developers, and they may take these suggestions into account if possible. For example, the developers may use these suggestions while choosing among different design solutions.

Examples of Bad SRS Documents

- **Unstructured Specifications:**
  - **Narrative essay --- one of the worst types of specification document:**
    - **Difficult to change,**
    - **Difficult to be precise,**
    - **Difficult to be unambiguous,**
    - **Scope for contradictions, etc.**

Examples of Bad SRS Documents

- **Noise:**
  - **Presence of text containing information irrelevant to the problem.**
- **Silence:**
  - **aspects important to proper solution of the problem are omitted.**

Examples of Bad SRS Documents

- **Overspecification:**
  - **Addressing "how to" aspects**
  - **For example, "Library member names should be stored in a sorted descending order"**
  - **Overspecification restricts the solution space for the designer.**
- **Contradictions:**
  - **Contradictions might arise**
    - **if the same thing described at several places in different ways.**

Examples of Bad SRS Documents

- Ambiguity:
  - Literary expressions
  - Unquantifiable aspects, e.g. "good user interface"
- Forward References:
  - References to aspects of problem
    - defined only later on in the text.
- Wishful Thinking:
  - Descriptions of aspects
    - for which realistic solutions will be hard to find.

Representation of complex processing logic:

- Decision trees
- Decision tables

Decision Trees

- Decision trees:
  - Edges of a decision tree represent conditions
  - Leaf nodes represent actions to be performed.
- A decision tree gives a graphic view of:
  - Logic involved in decision making
  - Corresponding actions taken.

Example: LMS

- A Library Membership automation Software (LMS) should support the following three options:
  - New member,
  - Renewal,
  - Cancel membership.

Example: LMS

- When the new member option is selected,
  - The software asks details about the member:
    - name,
    - address,
    - phone number, etc.

Example(cont.)

- If proper information is entered,
  - A membership record for the member is created
  - A bill is printed for the annual membership charge plus the security deposit payable.

Example(cont.)

- If the renewal option is chosen,
  - LMS asks the member's name and his membership number
    - checks whether he is a valid member.
  - If the name represents a valid member,
    - the membership expiry date is updated and the annual membership bill is printed,
    - otherwise an error message is displayed.

Example(cont.)

- If the cancel membership option is selected and the name of a valid member is entered,
  - The membership is cancelled,
  - A cheque for the balance amount due to the member is printed
  - The membership record is deleted.

Decision Tree

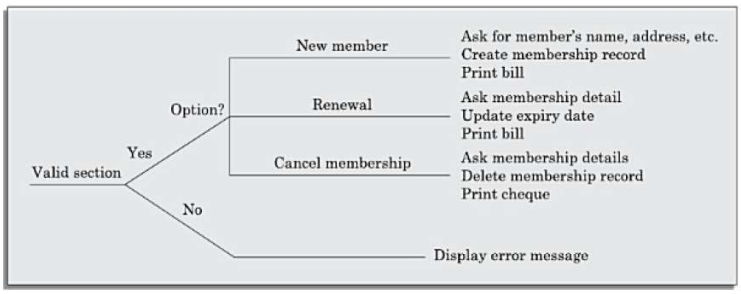


Figure 4.3: Decision Tree for LMS.

Decision Table

- Decision tables specify:
  - Which variables are to be tested
  - What actions are to be taken if the conditions are true,
  - The order in which decision making is performed.

Decision Table

- A decision table shows in a tabular form:
  - Processing logic and corresponding actions
- Upper rows of the table specify:
  - The variables or conditions to be evaluated
- Lower rows specify:
  - The actions to be taken when the corresponding conditions are satisfied.

Decision Table

- In technical terminology,
  - a column of the table is called a rule:
  - A rule implies:
    - if a condition is true, then execute the corresponding action.

Example:

Conditions				
Valid selection	NO	YES	YES	YES
New member	--	YES	NO	NO
Renewal	--	NO	YES	NO
Cancellation	--	NO	NO	YES
Actions				
Display error message	X			
Ask member's name etc.		X		
Build customer record		X		
Generate bill		X	X	
Ask membership details			X	X
Update expiry date			X	
Print cheque				X
Delete record				X

Comparison

- Both decision tables and decision trees
  - Can represent complex program logic.
- Decision trees are easier to read and understand
  - When the number of conditions are small.
- Decision tables help to look at every possible combination of conditions.

Formal Specification

- A formal specification technique is a mathematical method to:
  - Accurately specify a system.
  - Verify that implementation satisfies specification.
  - Prove properties of the specification.

Formal Specification

- Advantages:
  - Well-defined semantics, no scope for ambiguity
  - Automated tools can check properties of specifications
  - Executable specification

Formal Specification

- Disadvantages of formal specification techniques:
  - Difficult to learn and use
  - Not able to handle complex systems

Formal Specification

- Mathematical techniques used include:
  - Logic-based
  - set theoretic
  - algebraic specification
  - finite state machines, etc.

Semiformal Specification

- Structured specification languages
  - SADT (Structured Analysis and Design Technique)
  - PSL/PSA (Problem Statement Language/Problem Statement Analyzer)
    - PSL is a semi-formal specification language
    - PSA can analyze the specifications expressed in PSL

Executable Specification Language

- If specification is expressed in formal language:
  - it becomes possible to execute the specification to provide a system prototype.
- However, executable specifications are usually slow and inefficient.

Executable Specification Language

- Executable specifications only test functional requirements:
  - If non-functional requirements are important for some product,
    - The utility of an executable specification prototype is limited.

4GLs

- 4GLs (Fourth Generation Languages) are examples of
  - executable specification languages.
- 4GLs are successful
  - because there is a lot of commonality across data processing applications.

4GLs

- 4GLs rely on software reuse
  - Where common abstractions have been identified and parameterized.
- Rewriting 4GL programs in higher level languages:
  - Result in upto 50% lower memory requirements
  - Also the programs run upto 10 times faster.

Summary

- Requirements analysis and specification
  - An important phase of software development:
  - Any error in this phase would affect all subsequent phases of development.
- Consists of two different activities:
  - Requirements gathering and analysis
  - Requirements specification

Summary

- The aims of requirements analysis:
  - Gather all user requirements
  - Clearly understand exact user requirements
  - Remove inconsistencies and incompleteness.
- The goal of specification:
  - Systematically organize requirements
  - Document the requirements in an SRS document.

Summary

- Main components of SRS document:
  - Functional requirements
  - Non-functional requirements
  - Constraints
- Techniques to express complex logic:
  - Decision tree
  - Decision table

Summary

- Formal requirements specifications have several advantages.
  - But the major shortcoming is that these are hard to use.

Object-Orientation Concepts, UML, and OOAD

Organization of This Lecture

- Object-oriented concepts
- Object modelling using Unified Modelling Language (UML)
- Object-oriented software development and patterns
- CASE tools
- Summary

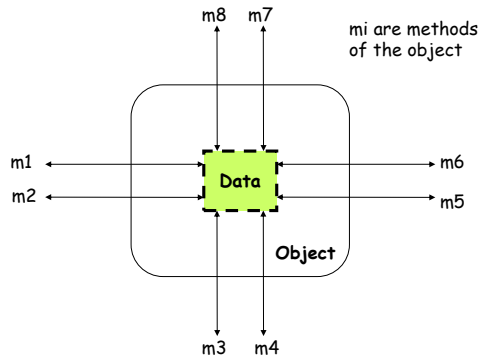
Object-Oriention Concepts

- Object-oriented (OO) design techniques are extremely popular:
  - Inception in early 1980's and nearing maturity.
  - Widespread acceptance in industry and academics.
  - Unified Modelling Language (UML) already an ISO standard (ISO/IEC 19501).

Objects

- A system is designed as a set of interacting objects:
  - Often, real-world entities:
    - Examples: an employee, a book etc.
  - Can be conceptual objects also:
    - Controller, manager, etc.
- Consists of data (attributes) and functions (methods) that operate on data.
  - Hides organization of internal information (Data abstraction).

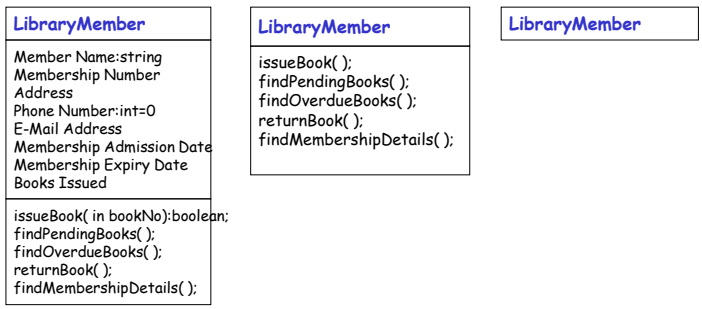
Model of an Object



Class

- Instances are objects
- Template for object creation
- Considered as abstract data type (ADT)
- Examples: Employees, Books, etc.
- Sometimes not intended to produce instances:
  - Abstract classes

Example Class Diagram



Different representations of the LibraryMember class

Methods and Messages

- Operations supported by an object:
  - Means for manipulating the data of other objects.
  - Invoked by sending a message (method call).
  - Examples: calculate_salary, issue-book, member_details, etc.

What are the Different Types of Relationships Among Classes?

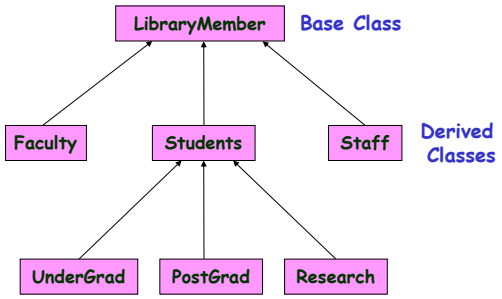
- Four types of relationships:
  - Inheritance
  - Association
  - Aggregation/Composition
  - Dependency

Inheritance

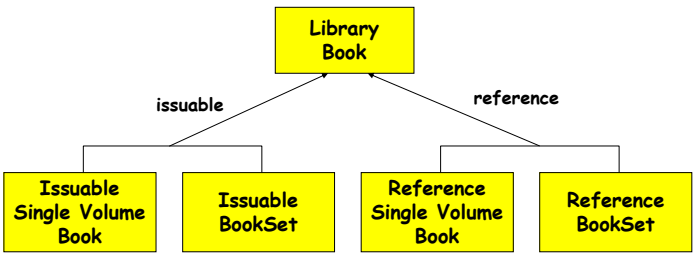
- Allows to define a new class (derived class) by extending or modifying existing class (base class).
  - Represents generalization-specialization relationship.
  - Allows redefinition of the existing methods (method overriding).

Inheritance

- Lets a subclass inherit attributes and methods from more than one base class.



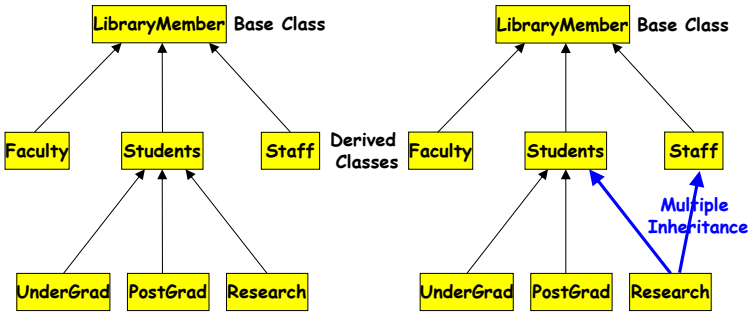
Inheritance Example



Representation of the inheritance relationship

Multiple Inheritance

cont…



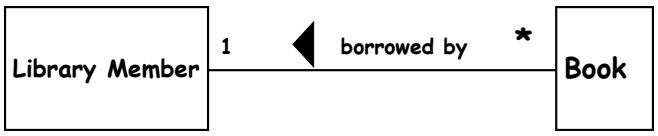
Association Relationship

- Enables objects to communicate with each other:
  - Thus one object must "know" the address of the corresponding object in the association.
- Usually binary:
  - But in general can be n-ary.

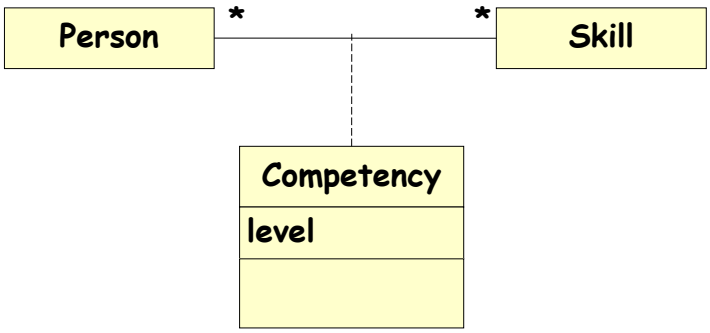
Association Relationship

- A class can be associated with itself (recursive association).
  - Give an example?
- An arrowhead used along with name, indicates direction of association.
- Multiplicity indicates # of instances taking part in the association.

Association Relationship



3-ary Association



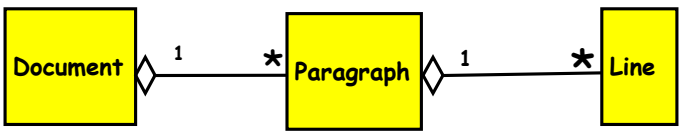
Association and Link

- A link:
  - An instance of an association
  - Exists between two or more objects
  - Dynamically created and destroyed as the run of a system proceeds
- For example:
  - An employee joins an organization,
  - Leaves that organization and joins a new organization etc.

Aggregation Relationship

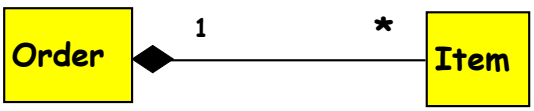
- Represents whole-part relationship
- Represented by a diamond symbol at the composite end
- Cannot be reflexive(i.e. recursive)
- Not symmetric
- It can be transitive

Aggregation Relationship



Composition Relationship

- Life of item is same as the order



Aggregation

cont…

- A aggregate object contains other objects.
- Aggregation limited to tree hierarchy:
  - No circular inclusion relation.

Aggregation vs. Inheritance

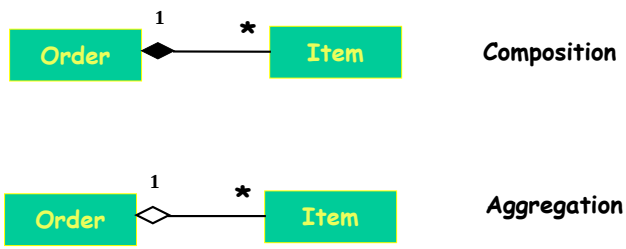
Cont…

- Inheritance:
  - Different object types with similar features.
  - Necessary semantics for similarity of behavior is in place.
- Aggregation:
  - Containment allows construction of complex objects.

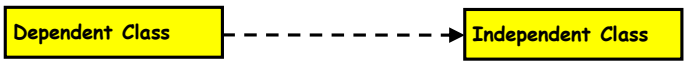
Aggregation vs. Composition

- Composition:
  - Composite and components have the same life.
- Aggregation:
  - Lifelines are different.
- Consider an order object:
  - Aggregation: If order items can be changed or deleted after placing the order.
  - Composition: Otherwise.

Composition versus Aggregation



Class Dependency



Representation of dependence between classes

Abstraction

- Consider aspects relevant for certain purpose
  - Suppress non-relevant aspects
- Types of abstraction:
  - Data abstraction
  - Behaviour abstraction

Abstraction

cont…

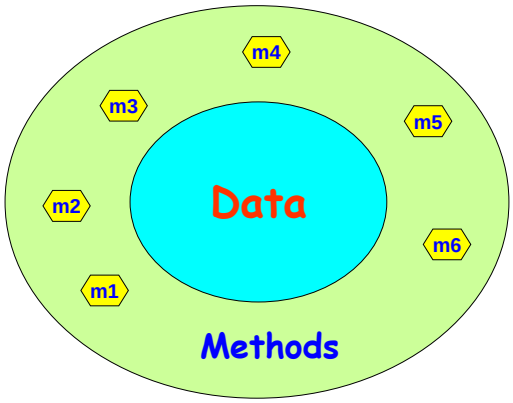
- Advantages of abstraction:
  - Reduces complexity of design
  - Enhances understandability
  - Increases productivity
- It has been observed that:
  - Productivity is inversely proportional to complexity.

Encapsulation

- Objects communicate with outside world through messages:
  - Data of objects encapsulated within its methods.
  - Data accessible only through methods.

Encapsulation

cont…



Concept of encapsulation

Polymorphism

- Denotes poly (many) morphism (forms).
- Under different situations:
  - Same message to the same object can result in different actions:
    - Static binding
    - Dynamic binding

An Example of Static Binding

- Class Circle{
  - private float x, y, radius;
  - private int fillType;
  -
  - public create ();
  - public create (float x, float y, float centre);
  - public create (float x, float y, float centre, int fillType);
  - }

An Example of Static Binding

cont…

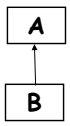
- A class named Circle has three definitions for create operation
  - Without any parameter, default
  - Centre and radius as parameter
  - Centre, radius and fillType as parameter
  - Depending upon parameters, method will be invoked
  - Method create is said to be overloaded

Dynamic Binding

- A method call to an object of an ancestor class:
  - Would result in the invocation of the method of an appropriate object of the derived class.
- Following principles are involved:
  - Inheritance hierarchy
  - Method overriding
  - Assignment to compatible types

Dynamic Binding

- Principle of substitutability (Liskov's substitutability principle):
  - An object can be assigned to an object of its ancestor class, but not vice versa.



A a; B b;

a=b; (OK)

b=a; (not OK)

Dynamic Binding

Cont …

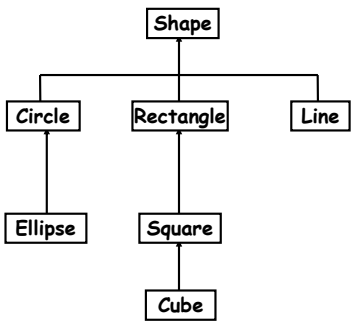
- Exact method to be bound on a method call:
  - Not possible to determine at compile time.
  - Dynamically decided at runtime.

An Example of Dynamic Binding

- Consider a class hierarchy of different geometric objects:
  - Display method is declared in the shape class and overridden in each derived class.
  - A single call to the display method for each object would take care of displaying the appropriate element.

An Example of Dynamic Binding

cont…



Class hierarchy of geometric objects

An Example

cont…

Traditional code	Object-oriented code

Shape s[1000];
For(i=0;i<1000;i++){
If (s[i] == Circle) then
draw_circle();
else if (s[i]== Rectangle) then
draw_rectangle();
-
 	
}
	Shape s[1000];
	For(i=0;i<1000;i++)
	Shape.draw();
	
	-
	-
	-
	-

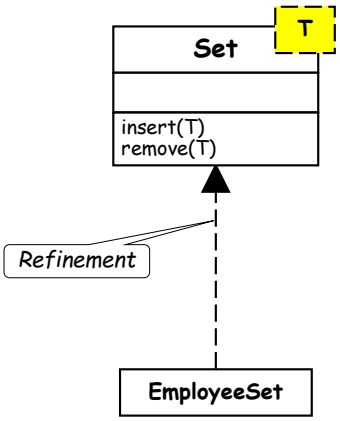
Traditional code and OO code using dynamic binding

Genericity

- Ability to parameterize class definitions.
- Example: class stack of different types of elements:
  - Integer stack
  - Character stack
  - Floating point stack
- Define generic class stack:
  - Later instantiate as required

```
TEMPLATE <TYPENAME T> T MYMAX(T X, T Y) {
    RETURN (X > Y) ? X : Y;
}
```


Genericity



Advantages of Object-Oriented Development

- Code and design reuse
- Increased productivity
- Ease of testing (?) and maintenance
- Better understandability
- Elegant design:
  - Loosely coupled, highly cohesive objects:
  - Essential for solving large problems.

Advantages of Object-Oriented Development cont…

- Initially incurs higher costs
  - After completion of some projects reduction in cost become possible
- Using well-established OO methodology and environment:
  - Projects can be managed with 20% -- 50% of traditional cost of development.

Object Modelling Using UML

- UML is a modelling language
  - Not a system design or development methodology
- Used to document object-oriented analysis and design results.
- Independent of any specific design methodology.

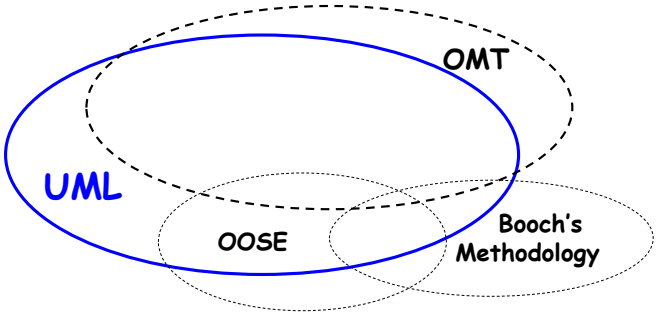
UML Origin

- OOD in late 1980s and early 1990s:
  - Different software development houses were using different notations.
  - Methodologies were tied to notations.
- UML developed in early 1990s to:
  - Standardize the large number of object-oriented modelling notations

UML Lineology

- Based Principally on:
  - **OMT** [Rumbaugh 1991]
  - **Booch's methodology**[Booch 1991]
  - **OOSE** [Jacobson 1992]
  - **Odell's methodology**[Odell 1992]
  - **Shlaer and Mellor** [Shlaer 1992]

Different Object Modeling Techniques in UML

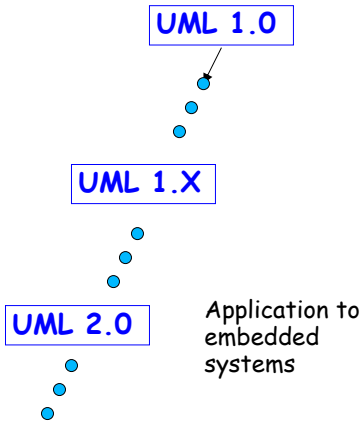


UML as A Standard

- Adopted by Object Management Group (OMG) in 1997
- OMG is an association of industries
- Promotes consensus notations and techniques
- Used outside software development
  - Example car manufacturing

Developments to UML

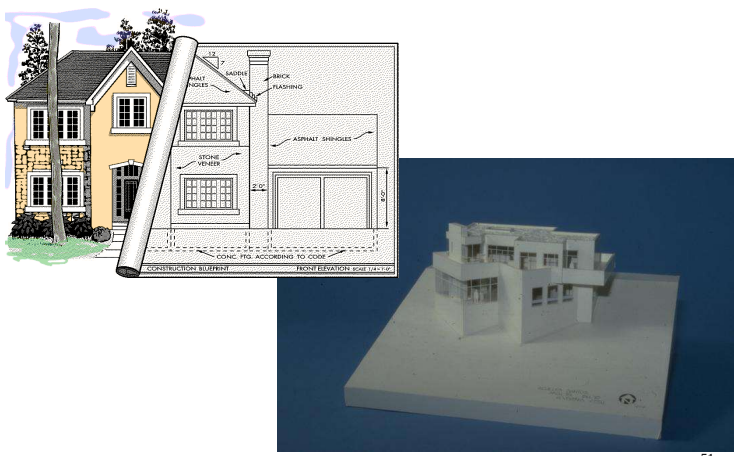
- **UML continues to develop:**
  - **Refinements**
  - **Making it applicable to new contexts**



Why are UML Models Required?

- A model is an abstraction mechanism:
  - Capture only important aspects and ignores the rest.
  - Different models result when different aspects are ignored.
  - An effective mechanism to handle complexity.
- UML is a graphical modelling tool
- Easy to understand and construct

Modeling a House



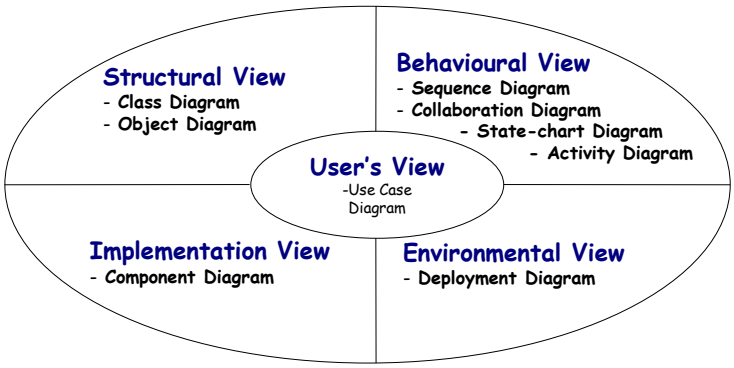
UML Diagrams

- Nine diagrams are used to capture different views of a system.
- Views:
  - Provide different perspectives of a software system.
- Diagrams can be refined to get the actual implementation of a system.

UML Model Views

- Views of a system:
  - User's view
  - Structural view
  - Behavioral view
  - Implementation view
  - Environmental view

UML Diagrams



Diagrams and views in UML

Are All Views Required for Developing A Typical System?

- **NO**
- Use case diagram, class diagram and one of the interaction diagram for a simple system
- State chart diagram required to be developed when a class state changes
- However, when states are only one or two, state chart model becomes trivial
- Deployment diagram in case of large number of hardware components used to develop the system

Use Case Model

- Consists of set of "use cases"
- An important analysis and design artifact
- The central model:
  - Other models must confirm to this model
  - Not really an object-oriented model
  - Represents a functional or process model

Use Cases

- Different ways in which a system can be used by the users
- Corresponds to the high-level requirements
- Represents transaction between the user and the system
- Defines external behavior without revealing internal structure of system
- Set of related scenarios tied together by a common goal.

Use Cases

Cont…

- Normally, use cases are independent of each other
- Implicit dependencies may exist
- **Example**: In Library Automation System, renew-book & reserve-book are independent use cases.
  - But in actual implementation of renew-book: a check is made to see if any book has been reserved using reserve-book.

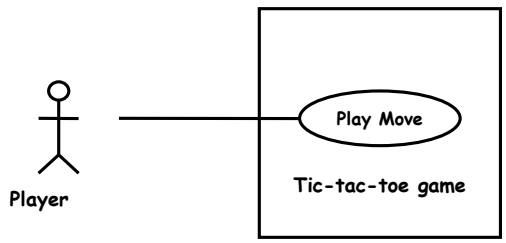
Example Use Cases

- For library information system
  - issue-book
  - query-book
  - return-book
  - create-member
  - add-book, etc.

Representation of Use Cases

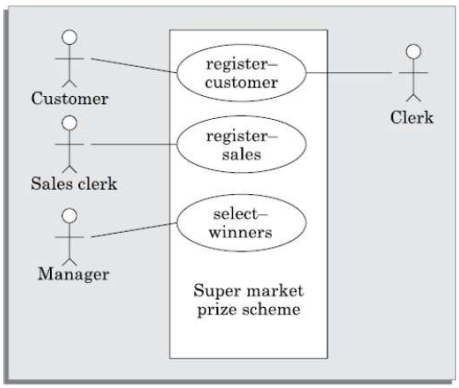
- Represented by use case diagram
- A use case is represented by an ellipse
- System boundary is represented by a rectangle
- Users are represented by stick person icons (actor)
- Communication relationship between actor and use case by a line
- External system by a stereotype

An Example Use Case Diagram



Use case model

Example Use Case



Example Use case

- **Text description**
- **U1: register-customer:** Using this use case, the customer can register himself by providing the necessary details.
- **Scenario 1: Mainline sequence**
- 1. Customer: select register customer option
- 2 . System: display prompt to enter name, address, and telephone number.
- 3. Customer: enter the necessary values
- 4: System: display the generated id and the message that the customer has successfully been registered.

- **Scenario 2: At step 4 of mainline sequence**

  4 : System: displays the message that the customer has already registered.

- **Scenario 3: At step 4 of mainline sequence**

  4 : System: displays message that some input information have not been entered. The system displays a prompt to enter the missing values.

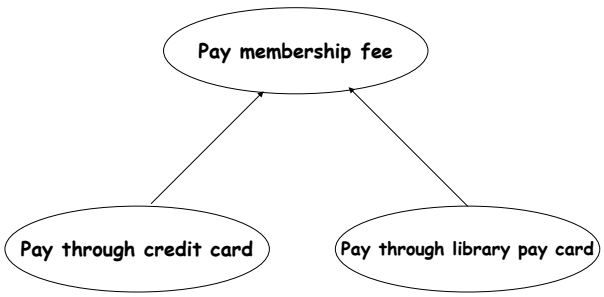
Why Develop A Use Case Diagram?

- Serves as requirements specification
- How are actor identification useful in software development:
  - User identification helps in implementing appropriate interfaces for different categories of users
  - Another use in preparing appropriate documents (e.g. **user's manual**).

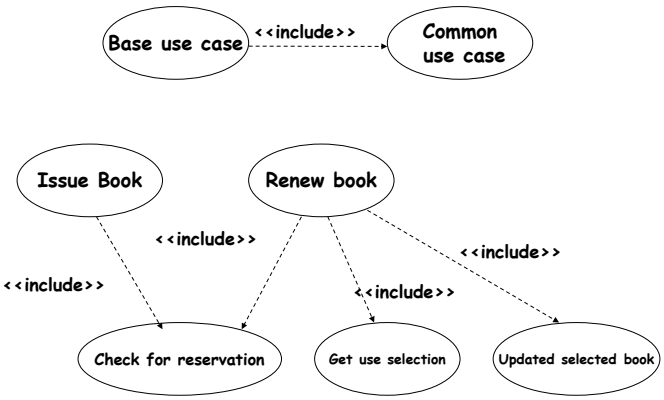
Factoring Use Cases

- Two main reasons for factoring:
  - Complex use cases need to be factored into simpler use cases
  - To represent common behavior across different use cases
- Three ways of factoring:
  - Generalization
  - Includes
  - Extends

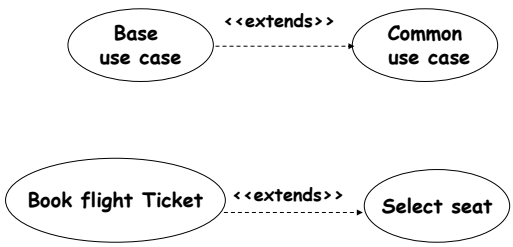
Factoring Use Cases Using Generalization



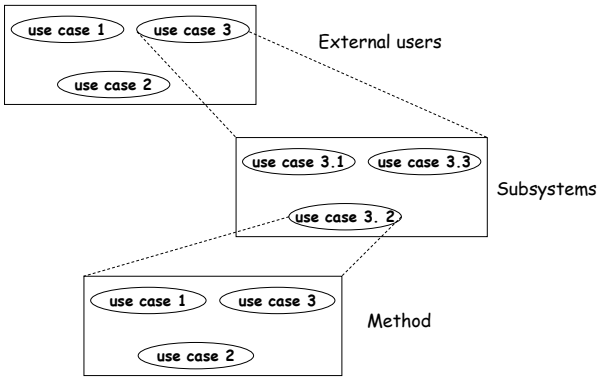
Factoring Use Cases Using Includes



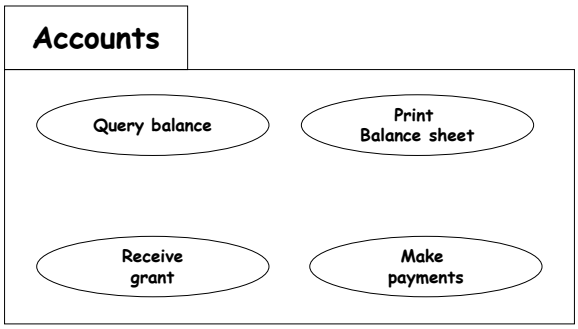
Factoring Use Cases Using Extends



Hierarchical Organization of Use Cases



Use Case Packaging



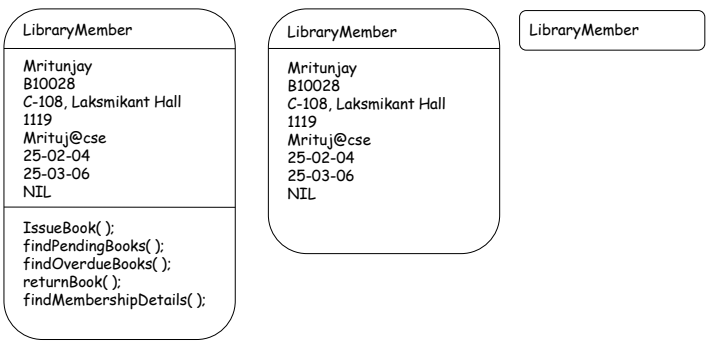
Class Diagram

- Describes static structure of a system
- Main constituents are classes and their relationships:
  - Generalization
  - Aggregation
  - Association
  - Various kinds of dependencies

Class Diagram

- Entities with common features, i.e. attributes and operations
- Classes are represented as solid outline rectangle with compartments
- Compartments for name, attributes, and operations.
- Attribute and operation compartments are optional depending on the purpose of a diagram.

Object Diagram



Different representations of the LibraryMember object

Interaction Diagram

- Models how groups of objects collaborate to realize some behaviour
- Typically each interaction diagram realizes behaviour of a single use case

Interaction Diagram

- Two kinds: Sequence and Collaboration diagrams.
- Two diagrams are equivalent
  - Portray different perspectives
- These diagrams play a very important role in the design process.

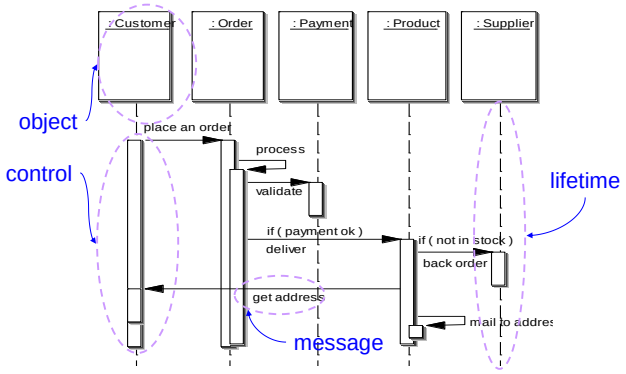
Sequence Diagram

- Shows interaction among objects as a two-dimensional chart
- Objects are shown as boxes at top
- If object created during execution then shown at appropriate place
- Objects existence are shown as dashed lines (lifeline)
- Objects activeness, shown as a rectangle on lifeline

Sequence Diagram Cont…

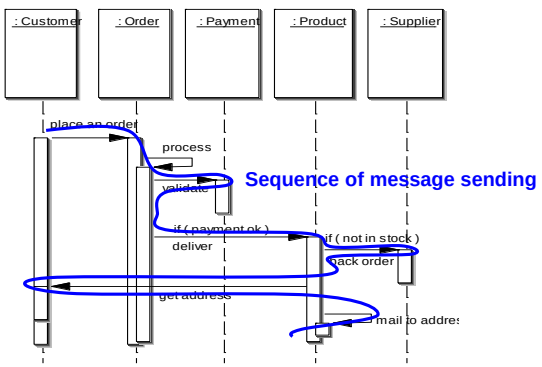
- Messages are shown as arrows
- Each message labelled with corresponding message name
- Each message can be labelled with some control information
- Two types of control information
  - condition ([])
  - iteration (*)

Elements of a Sequence Diagram

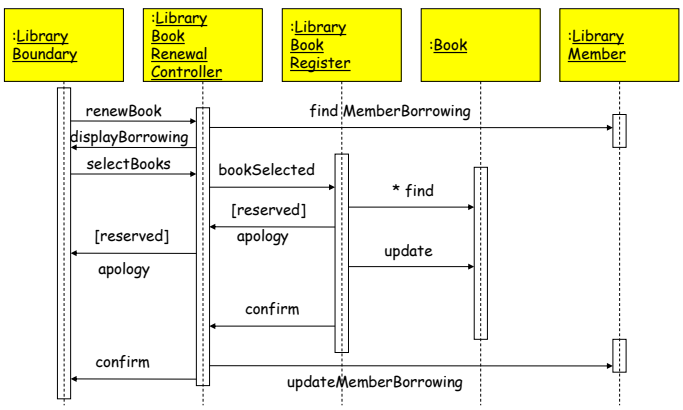


Example

Cont...



An Example of A Sequence Diagram

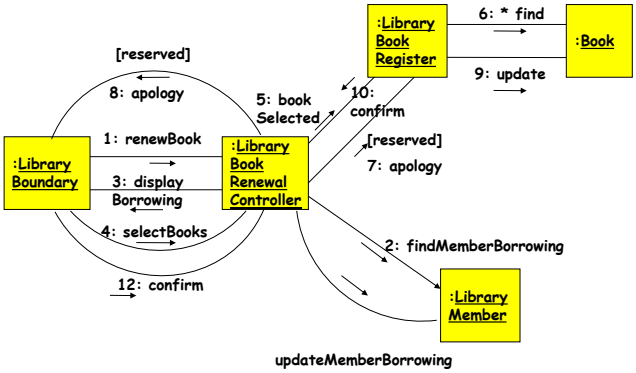


Sequence Diagram for the renew book use case

Collaboration Diagram

- Shows both structural and behavioural aspects
- Objects are collaborator, shown as boxes
- Messages between objects shown as a solid line
- A message is shown as a labelled arrow placed near the link
- Messages are prefixed with sequence numbers to show relative sequencing

An Example of A Collaboration Diagram



Collaboration Diagram for the renew book use case

Activity Diagram

- Not present in earlier modelling techniques:
  - Possibly based on event diagram of Odell [1992]
- Represents processing activity, may not correspond to methods
- Activity is a state with an internal action and one/many outgoing transitions
- Somewhat related to flowcharts

Activity Diagram vs Flow Chart

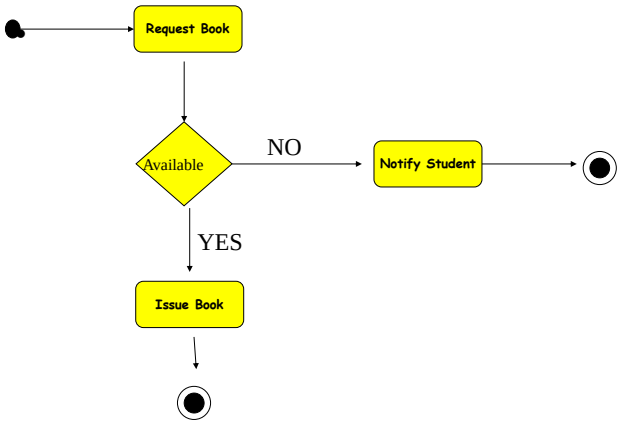
- Can represent parallel activity and synchronization aspects
- **Swim lanes** can be used to group activities based on who is performing them
- Example: academic department vs. hostel

Activity Diagram

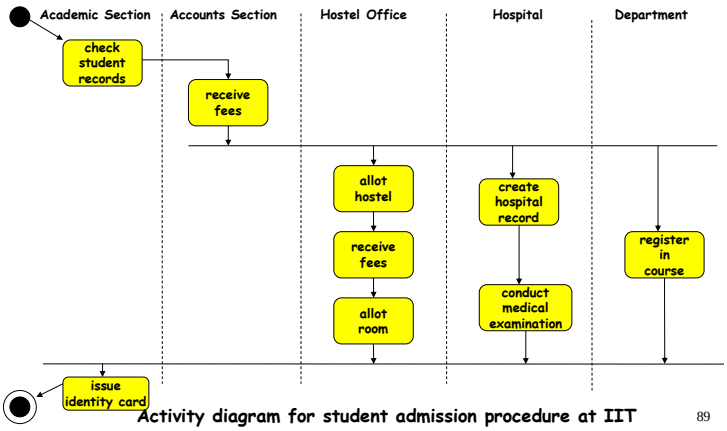
- Normally employed in business process modelling.
- Carried out during requirements analysis and specification stage.
- Can be used to develop interaction diagrams.

•**Initial Node (Start Point)** → Black filled circle ●
Marks the beginning of the workflow.
•**Activity / Action** → Rounded rectangle ☐ with curved edges
Represents a step in the process.
•**Control Flow (Arrow)** → →
Shows the order of execution.
•**Decision Node** → Diamond ♦
Used for **branching**
•**Merge Node** → Diamond ♦
Brings multiple alternate flows back together.
•**Fork Node** → Thick horizontal/vertical bar
Splits one flow into **parallel activities**.
•**Join Node** → Thick horizontal/vertical bar
Combines multiple parallel flows into one.
•**Final Node (End Point)** → Circle with a dot ⊙
Marks the end of the workflow.

An Example of An Activity Diagram

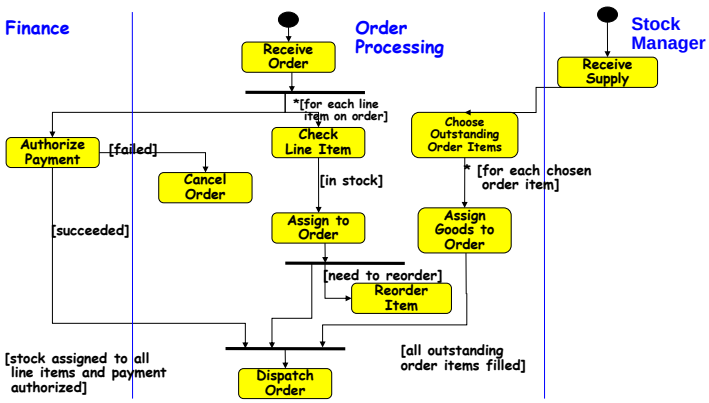


An Example of An Activity Diagram



Activity diagram for student admission procedure at IIT

Activity Diagram: Example 2



State Chart Diagram

- Based on the work of **David Harel** [1990]
- Model how the state of an object changes in its lifetime
- Based on finite state machine (FSM) formalism

State Chart Diagram

Cont…

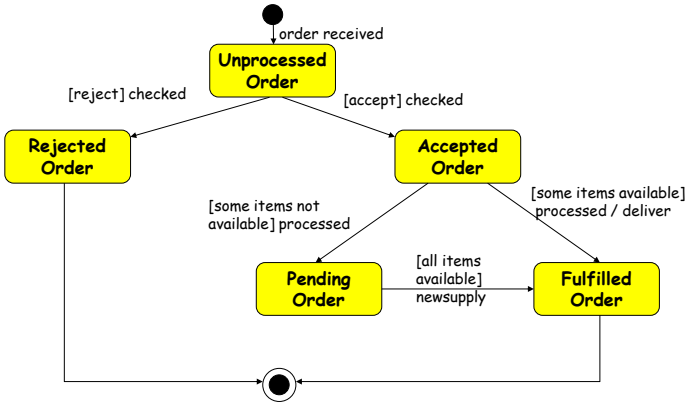
- State chart avoids the problem of state explosion of FSM.
- Hierarchical model of a system:
  - Represents composite **nested** states

State Chart Diagram

Cont…

- Elements of state chart diagram
- Initial State: A filled circle
- Final State: A filled circle inside a larger circle
- State: Rectangle with rounded corners
- Transitions: Arrow between states, also boolean logic condition (guard)

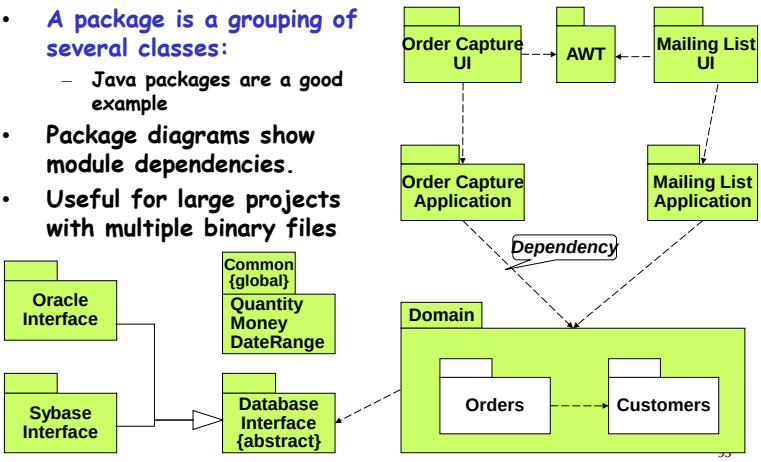
An Example of A State Chart Diagram



Example: State chart diagram for an order object

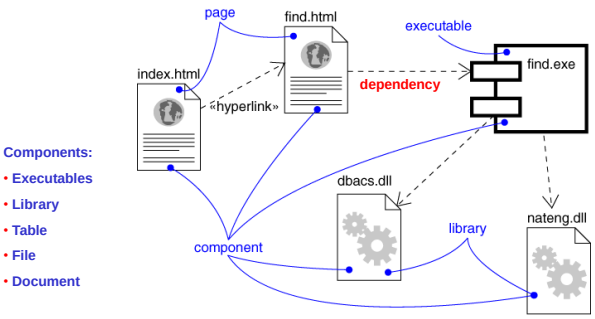
Package Diagrams

- A package is a grouping of several classes:
  - Java packages are a good example
- Package diagrams show module dependencies.
- Useful for large projects with multiple binary files



Component Diagram

- Captures the physical structure of the implementation (code components)

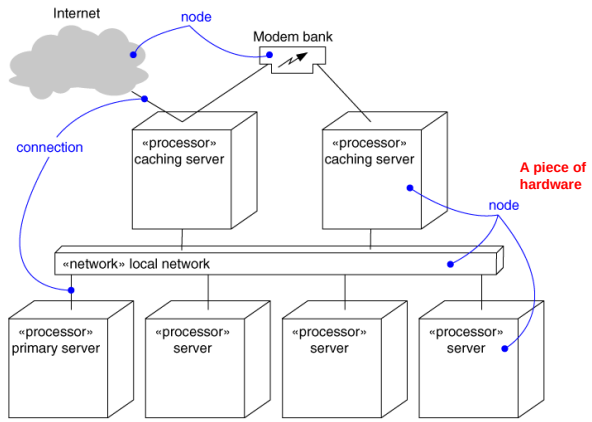


Component Diagram

- Captures the physical structure of the implementation
- Built as part of architectural specification
- Purpose
  - Organize source code
  - Construct an executable release
  - Specify a physical database
- Developed by architects and programmers

Deployment Diagram

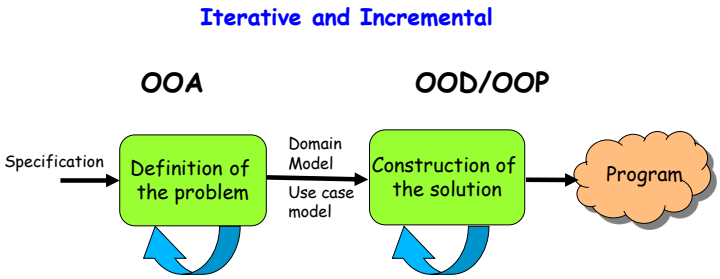
- Captures the topology of a system's hardware



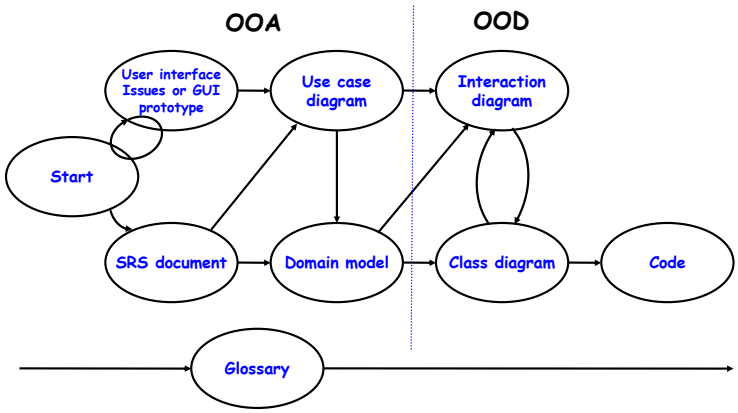
A Design Process

- Developed from various methodologies.
  - However, UML has been designed to be usable with any design methodology.
- From requirements specification, initial model is developed (OOA)
  - Analysis model is iteratively refined into a design model
- Design model is implemented using OO concepts

OOAD



Unified Development Process Cont…

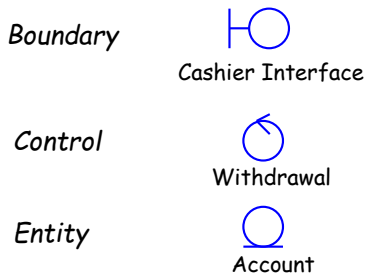


Domain Modelling

- Represents concepts or objects appearing in the problem domain.
- Also captures relationships among objects.
- Three types of objects are identified
  - Boundary objects
  - Entity objects
  - Controller objects

Class Stereotypes

Three different stereotypes on classes are used: <<boundary>>, <<control>>, <<entity>>.



Boundary Objects

- Interact with actors:
  - User interface objects
- Include screens, menus, forms, dialogs etc.
- Do not perform processing but validates, formats etc.

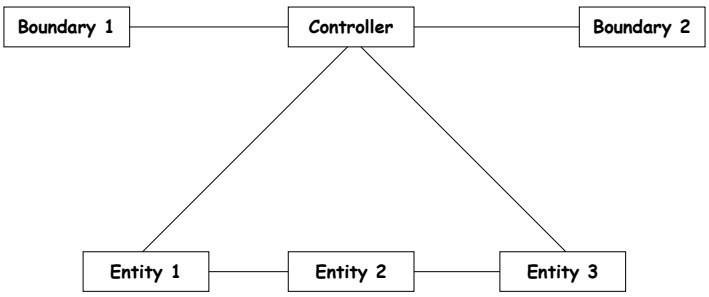
Entity Objects

- Hold information:
  - Such as data tables & files, e.g. Book, BookRegister
- Normally are dumb servers
- Responsible for storing data, fetching data etc.
- Elementary operations on data such as searching, sorting, etc.
- Entity Objects are identified by examining nouns in problem description

Controller Objects

- Coordinate the activities of a set of entity objects
- Interface with the boundary objects
- Realizes use case behavior
- Embody most of the logic involved with the use case realization
- There can be more than one controller to realize a single use case

Use Case Realization



Realization of use case through the collaboration of Boundary, controller and entity objects

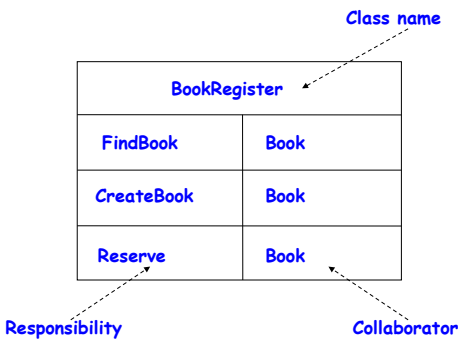
Class-Responsibility-Collaborator(CRC) Cards

- Pioneered by Ward Cunningham and Kent Beck
- Index cards prepared one each per class
- Class responsibility is written on these cards
- Collaborating object is also written

CRC Cards Cont…

- Required for developing interaction diagram of complex use cases
- Team members participate to determine:
  - The responsibility of classes involved in the use case realization

An Example of A CRC Card



CRC card for the BookRegister class

Patterns versus Idioms

- A pattern:
  - Describes a recurring problem
  - Describes the *core* of a solution
  - Is capable of generating many distinct designs
- An Idiom is more restricted
  - Still describes a recurring problem
  - Provides a more specific solution, with fewer variations
  - Applies only to a narrow context
    - e.g., the C++ language

Patterns

- **The essential idea:**
  - **If you can master a few important patterns, you can easily spot them in application development and use the pattern solutions.**

Idioms

- In English:
  - A group of words that has meaning different from a simple juxtaposition of the meanings of the individual words.
  - "Raining cats and dogs"
- A C idiom:
  - for(i=0;i<1000;i++){
  - }

Antipattern

- If a pattern represents a best practice:
  - Antipattern represents lessons learned from a bad design.
- Antipatterns help to recognise deceptive solutions:
  - That appear attractive at first, but turn out to be a liability later.

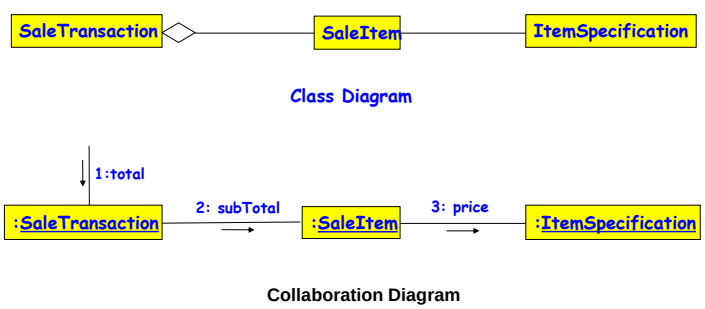
Design Patterns

- Standard solutions to commonly recurring problems
- Provides good solution based on common sense
- Pattern has four important parts
  - The problem
  - The context
  - The solution
  - The context in which it works or does not work

Example Pattern: Expert

- **Problem**: Which class should be responsible for doing certain things
- **Solution**: Assign responsibility to the class that has the information necessary to fulfil the required responsibility

Example Pattern: Expert Cont…



Example Pattern: Creator

- Problem: Which class should be responsible for creating a new instance of some class?
- Solution: Assign a class C1 the responsibility to create class C2 if
  - C1 is an aggregation of objects of type C2
  - C1 contains object of type C2

Example Pattern: Controller

- Problem: Who should be responsible for handling the actor requests?
- Solution: Separate controller object for each use case.

Example Pattern: Facade

- Problem: How should the services be requested from a service package?
- Context (problem): A package (cohesive set of classes), example: RDBMS interface package
- Solution: A class (DBfacade) can be created which provides a common interface to the services of the package

Example Pattern: MVC

- Model-View-Controller
- How should the user interface (Boundary) objects interact with the other objects?
- Solution 1: Pull from Above
  - Boundary object invokes other objects.
  - Does not work when data needs to be asynchronously displayed, simulation experiment, stock market alert, network monitor, etc.

Example Pattern: MVC

- Solution 2: Publish-Subscribe
  - The boundary objects register themselves with an event manager object.
  - Other objects, notify the event manager object as and when an event of interest occurs.
  - The event manager notifies those boundary objects that have registered with it by using a call back.

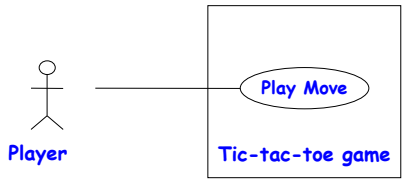
Example 1: Tic-Tac-Toe Computer Game

- A human player and the computer make alternate moves on a 3 3 square.
- A move consists of marking a previously unmarked square.
- The user inputs a number between 1 and 9 to mark a square
- Whoever is first to place three consecutive marks along a straight line (i.e., along a row, column, or diagonal) on the square wins.

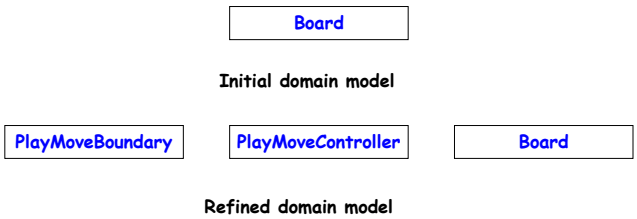
Example 1: Tic-Tac-Toe Computer Game

- As soon as either of the human player or the computer wins,
  - A message announcing the winner should be displayed.
- If neither player manages to get three consecutive marks along a straight line,
  - And all the squares on the board are filled up,
  - Then the game is drawn.
- The computer always tries to win a game.

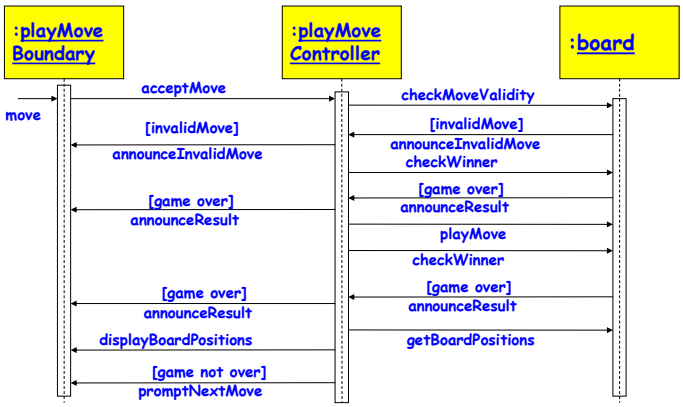
Example 1: Use Case Model



Example 1: Initial and Refined Domain Model

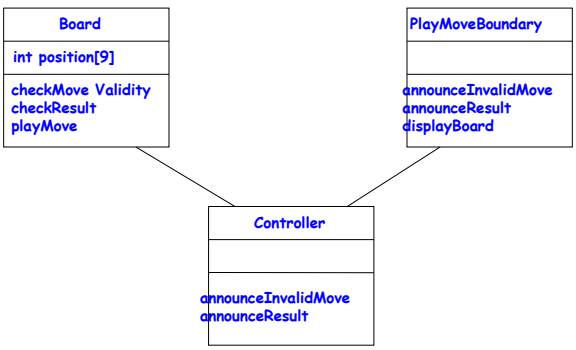


Example 1: Sequence Diagram



Sequence Diagram for the play move use case

Example 1: Class Diagram



Example 2: Supermarket Prize Scheme

- Supermarket needs to develop software to encourage regular customers.
- Customer needs to supply his:
  - Residence address, telephone number, and the driving licence number.
- Each customer who registers is:
  - Assigned a unique customer number (CN) by the computer.

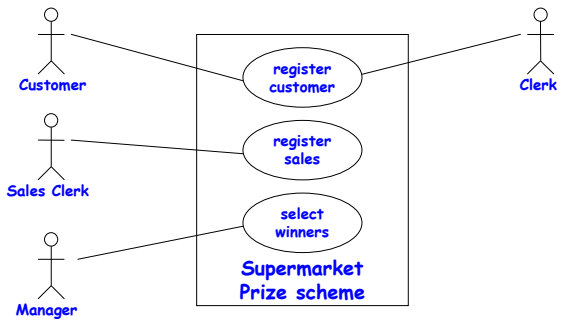
Example 2: Supermarket Prize Scheme

- A customer can present his CN to the staff when he makes any purchase.
- The value of his purchase is credited against his CN.
- At the end of each year:
  - The supermarket awards surprise gifts to ten customers who make highest purchase.

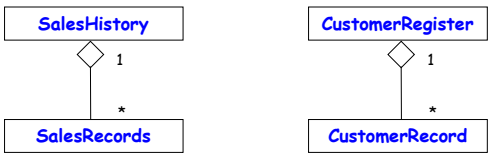
Example 2: Supermarket Prize Scheme

- Also, it awards a 22 carat gold coin to every customer:
  - Whose purchases exceed Rs. 10,000.
- The entries against the CN are reset:
  - On the last day of every year after the prize winner's lists are generated.

Example 2: Use Case Model

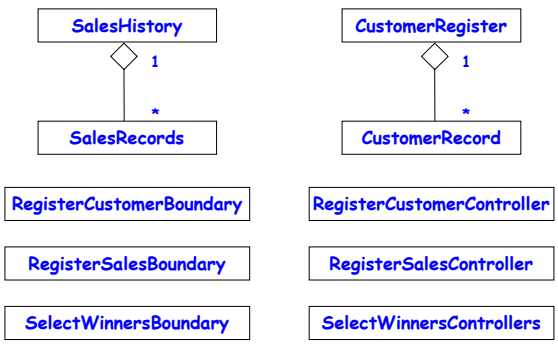


Example 2: Initial Domain Model



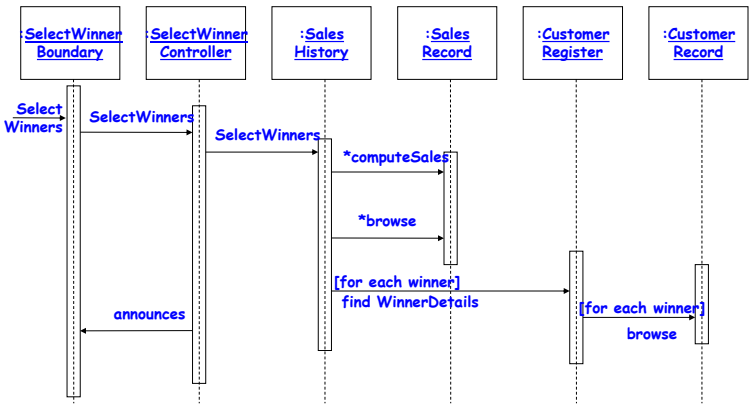
Initial domain model

Example 2: Refined Domain Model



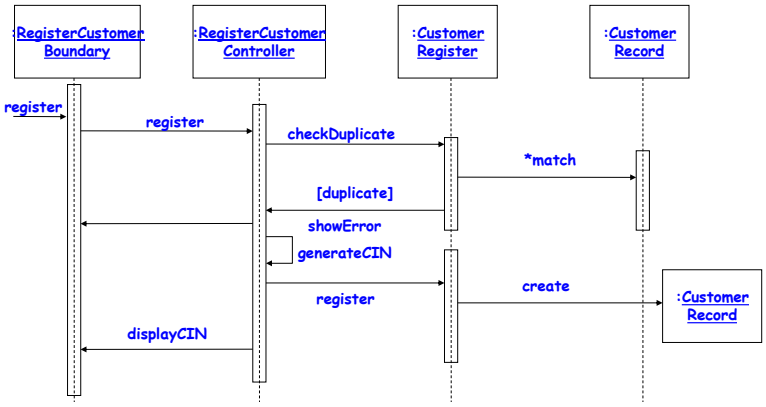
Refined domain model

Example 2: Sequence Diagram for the Select Winners Use Case



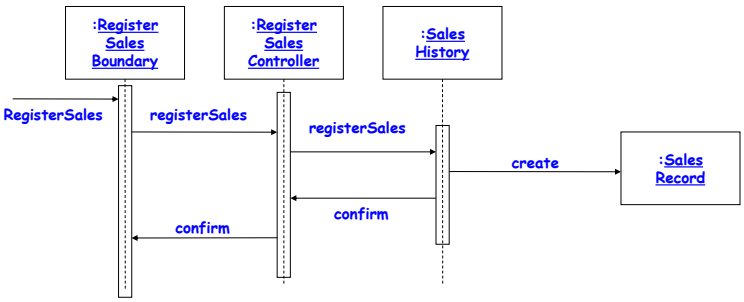
Sequence Diagram for the select winners use case

Example 2: Sequence Diagram for the Register Customer Use Case



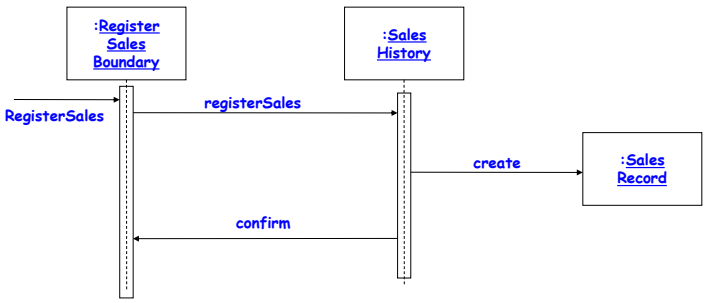
Sequence Diagram for the register customer use case

Example 2: Sequence Diagram for the Register Sales Use Case



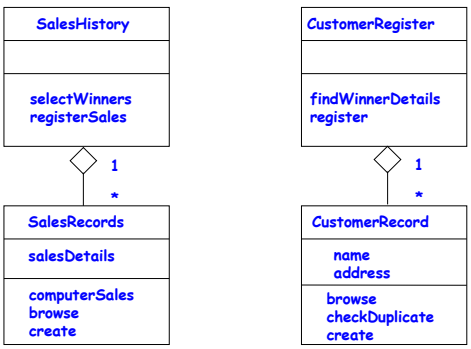
Sequence Diagram for the register sales use case

Example 2: Sequence Diagram for the Register Sales Use Case



Refined Sequence Diagram for the register sales use case

Example 2: Class Diagram



Summary

- We discussed object-oriented concepts
  - Basic mechanisms: Such as objects, class, methods, inheritance etc.
  - Key concepts: Such as abstraction, encapsulation, polymorphism, composite objects etc.

Summary

- We discussed an important OO language UML:
  - Its origin, as a standard, as a model
  - Use case representation, its factorisation such as generalization, includes and extends
  - Different diagrams for UML representation
  - In class diagram we discussed some relationships association, aggregation, composition and inheritance

Summary

cont…

- Other UML diagrams:
  - Interaction diagrams (sequence and collaboration),
  - Activity diagrams,
  - State chart diagrams.
- We discussed OO software development process:
  - Use of patterns lead to increased productivity and good solutions.